

BEYOND THE INTERVIEWS

System Design Interview Vault

Caching, CAP, consistent hashing and the patterns interviewers probe.

7 chapters · compiled from the Evolving Engineer — pradyumnachippigiri.substack.com

By Pradyumna Chippigiri

Compiled 4 October 2026

WHAT'S INSIDE

- Caching Playbook for System Design Interviews
- A Simple Proof of the CAP Theorem
- Easy Explanation of Consistent Hashing
- Concurrency Controls : Pessimistic vs Optimistic
- Client-Side Proxy vs Server-Side Proxy
- The Power of Bloom Filters in System Design
- Apache Kafka Basics

A Simple Proof of the CAP Theorem	15
<i>Everything about caches - layers, strategies, eviction policies, and production edge cases.</i>	
Caching Playbook for System Design Interviews	3

CONTENTS

Caching Playbook for System Design Interviews

2026-01-10 · Everything about caches - layers, strategies, eviction policies, and production edge cases.

Source: <https://pradyumnachippigiri.substack.com/p/caching-playbook-for-system-design>

Appreciate you reading this newsletter, I am writing as I learn too. I hope the content and resources are helping you level up as an engineer. If you want me to write about something specific, just message me or comment on the post.

Note: This article on was originally published on my personal website

-<https://pradyumnachippigiri.dev/blogs/caching-strategies>

Here's what I learned this week

2-random cache eviction

Ways to load images faster on web pages

llms.txt endpoint

What is Caching ??

Caching is basically keeping a copy of data (or a computed result) in a faster place so future requests can be served quickly without doing the “expensive work” again (DB query, disk read, network call, heavy computation).

The main goal of a cache is to reduce the application latency and the load on the database.

Caching prevents repeated disk I/O, network I/O, and CPU-intensive computation, which is what makes it such a powerful lever for scaling read traffic.

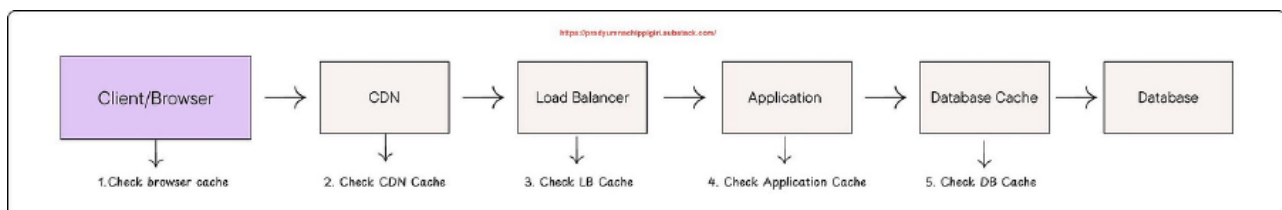
Caches are super fast but also expensive and limited in size, hence we don't cache everything. Instead, we strategically cache a subset of the data (hot data) that is most likely to be requested again.

What to Cache	What not to Cache
Read-heavy data (frequently read, infrequently updated)	Highly dynamic data that changes frequently
Expensive computations (CPU-heavy results)	User-specific sensitive data without proper isolation
Database query results (stable, repeatable queries)	Data requiring strong consistency guarantees
API responses (third-party calls with consistent data)	Data with low hit rates (rarely accessed)
Rendered HTML pages (static or semi-static content)	
Session data (auth/session state)	
Static assets (images, CSS, JavaScript)	

Caching can be done at multiple places in an application. Let's look at them below.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Caching at different levels



While these five layers represent the physical locations where data is stored, we can categorize them into four broad domains based on who manages the infrastructure and how the data is shared:

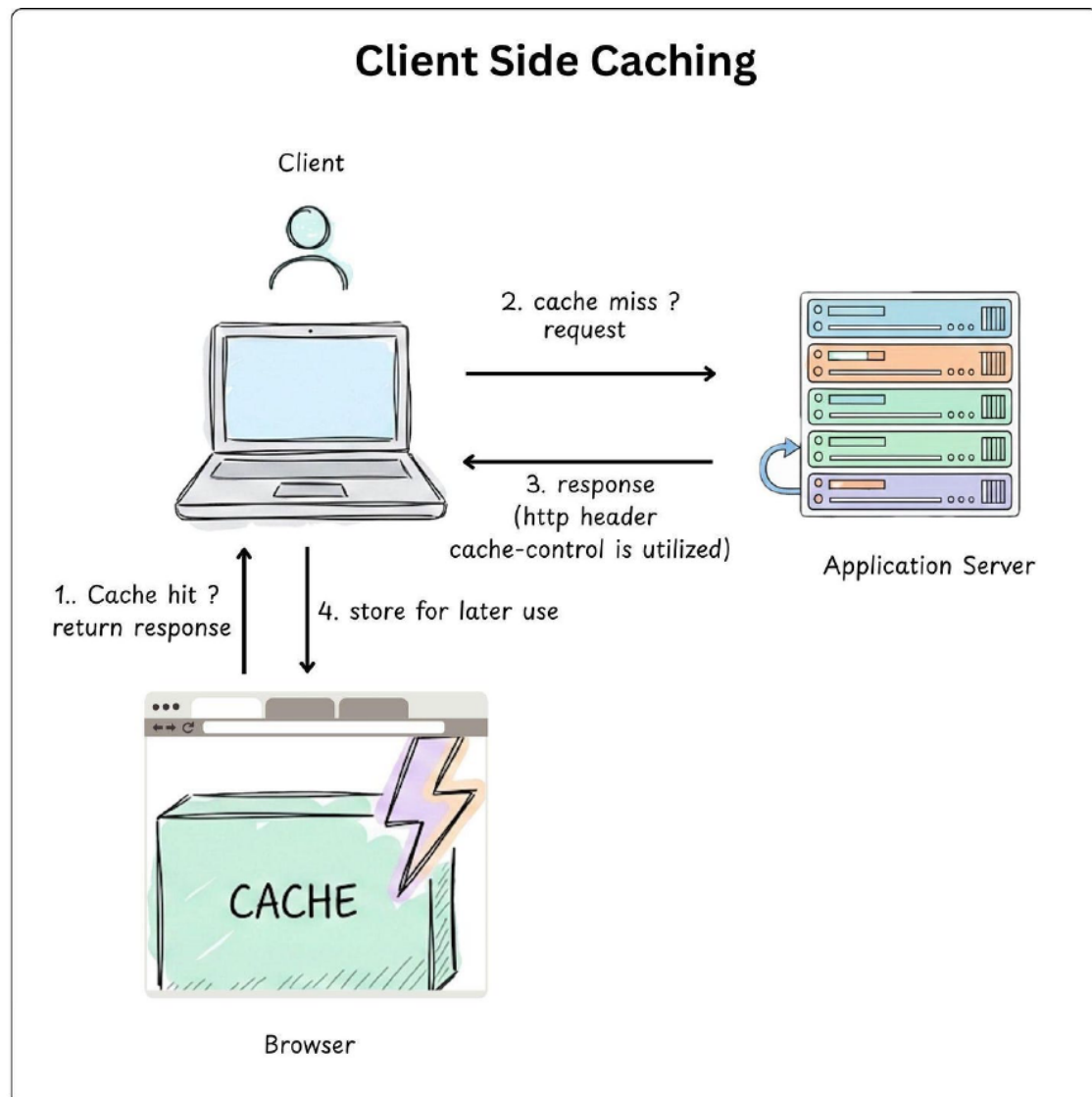
Client-Side Caching

Covers the browser cache; Store site-specific files (logo, CSS, JS) on the user's device. Hence provides with the lowest possible latency.

backend control is limited and invalidation can be harder.

Uses HTTP headers like Cache-Control, ETag, Expires, and Last-Modified.

Best for static assets (logo, CSS, JS) that don't change often.



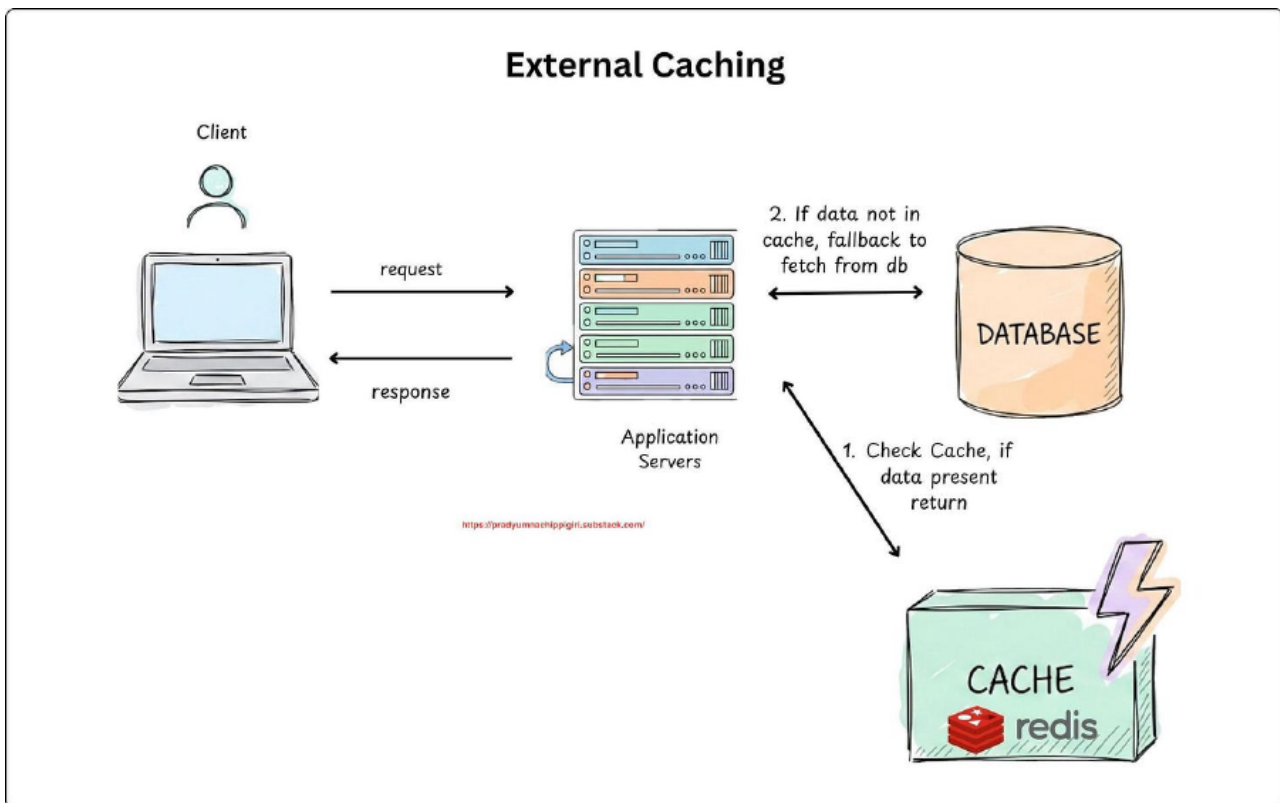
External Caching

It's a standalone service your app talks to. It acts as a shared memory for your entire all of our backend servers.

Can horizontally scale well, as all application servers share the same cache (meaning cache services are decoupled from backend services.)

Mostly used external cache are Redis, Memcached etc.

Our backend has control over TTL, Cache invalidation and Cache eviction policies.

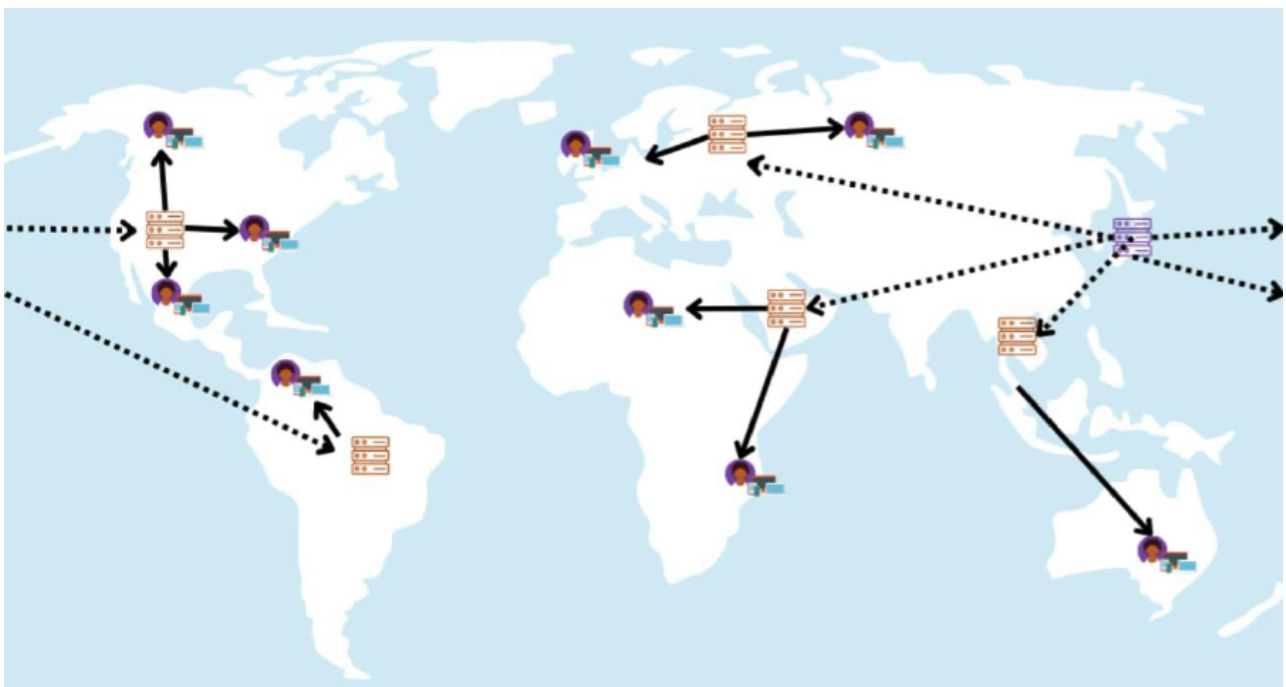


CDN

Content Delivery Network is like a globally distributed network of servers that caches copies of our content at the edge locations instead of burdening the origin server.

Shorter distance means faster delivery and less load on origin.

Mostly best for static content like images, videos etc.

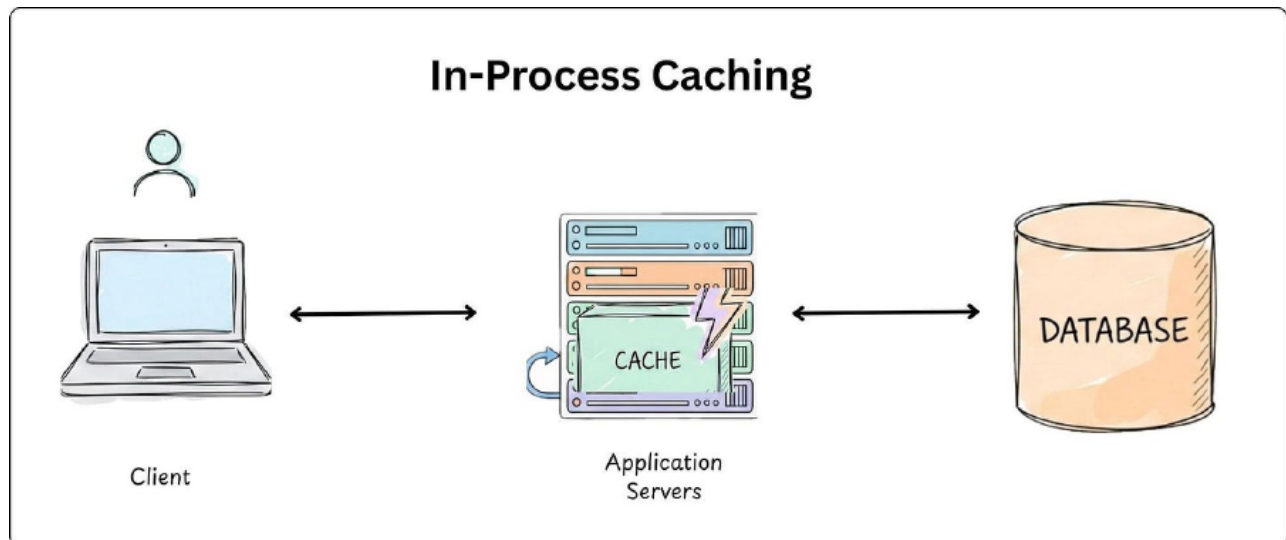


In-Process Caching

This is caching data directly inside your server's RAM (e.g., a local Hashmap).

It's blazing fast (zero network call) but specific to that one server instance. If one instance updates a cache, the others won't know.

Useful for small, frequently requested data like configuration values, feature flags, and precomputed values.

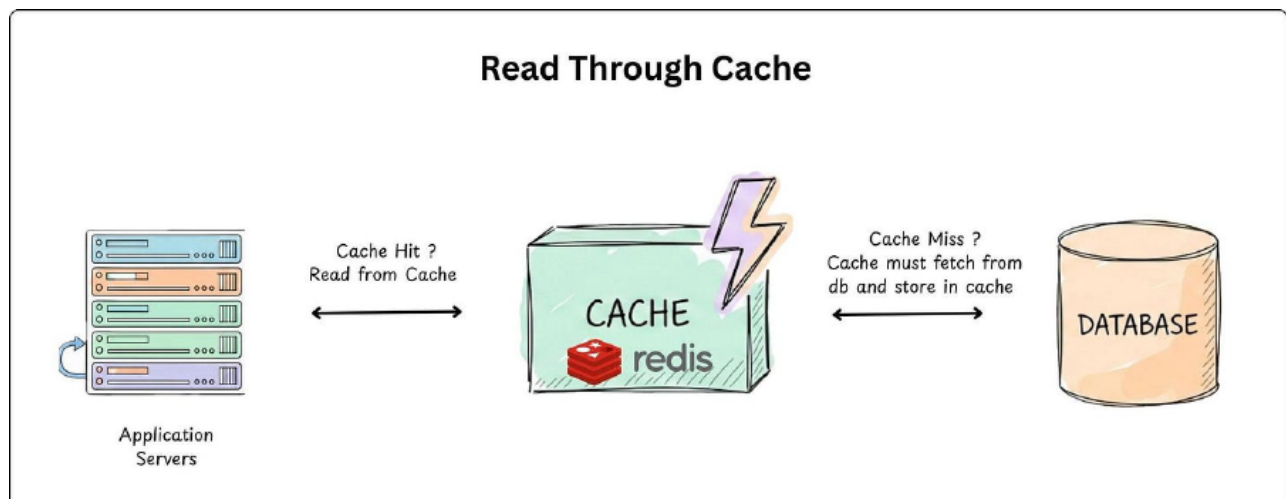


Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Caching Strategies

There are 4 core caching strategies in external caching that every engineer should know. And their use-cases entirely depend on what the application and its traffic demands.

Read Through



THE FLOW

Application requests data specifically from the cache.

On a miss, the cache fetches from the DB, stores it, and returns it.

LOGIC & OWNERSHIP

The cache acts as an intermediary between the app and DB. The cache handles reading and storing automatically.

BENEFITS

Simplifies application logic.

Low-latency data access for cache hits.

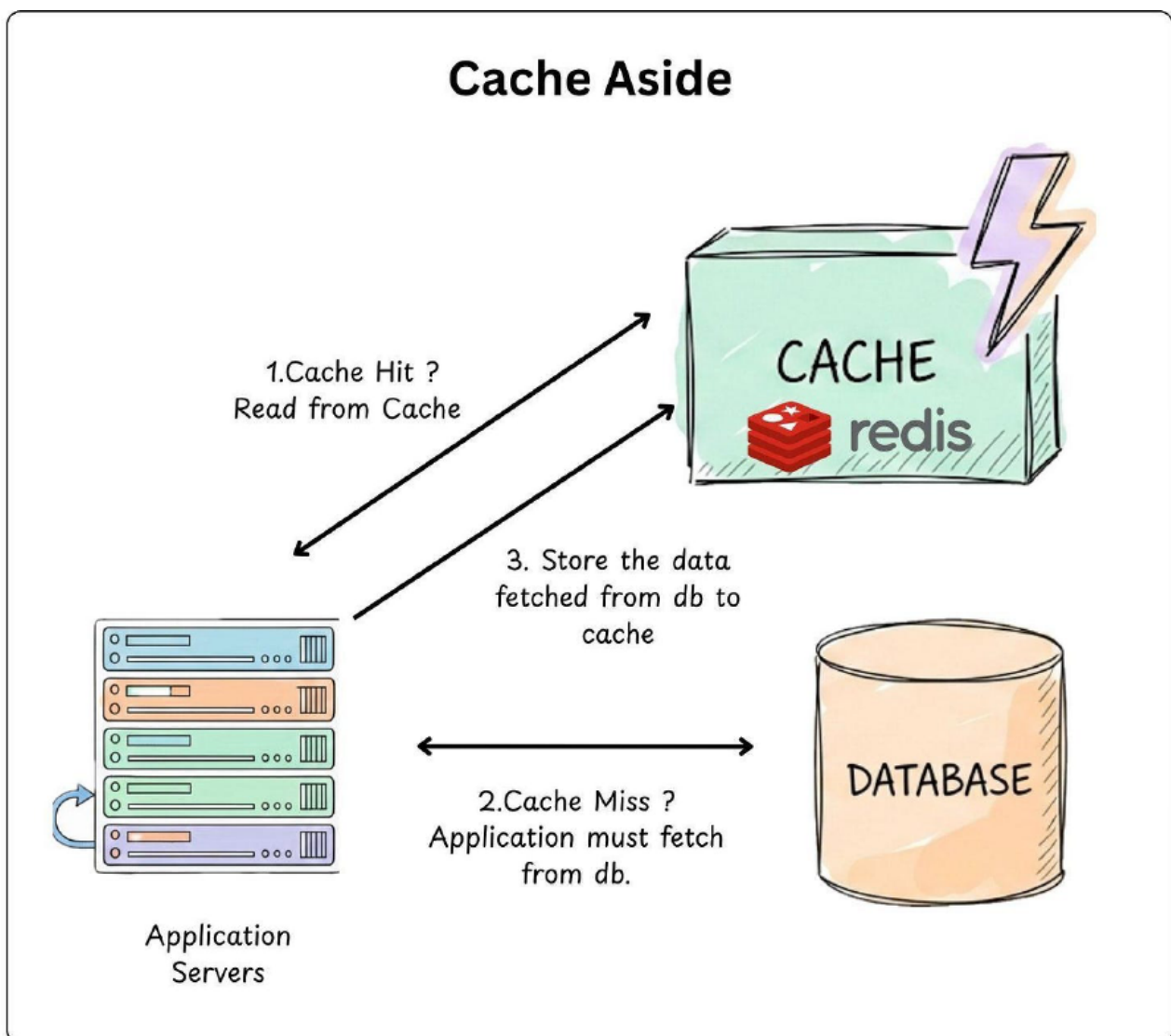
TRADE-OFFS

Higher latency during initial reads due to the mandatory DB query.

BEST FOR:

Read-heavy apps like CDNs or social media feeds.

Cache Aside (Lazy Loading)



THE FLOW

Application checks the cache first.

On a miss, the app retrieves from DB and manually loads it into the cache.

LOGIC & OWNERSHIP

Application code manages the interaction between cache and DB. Data is only loaded into the cache when needed (Lazy Loading).

BENEFITS

Highly resilient; if the cache fails, the app still functions via the DB.

Only “hot” data is stored in the cache.

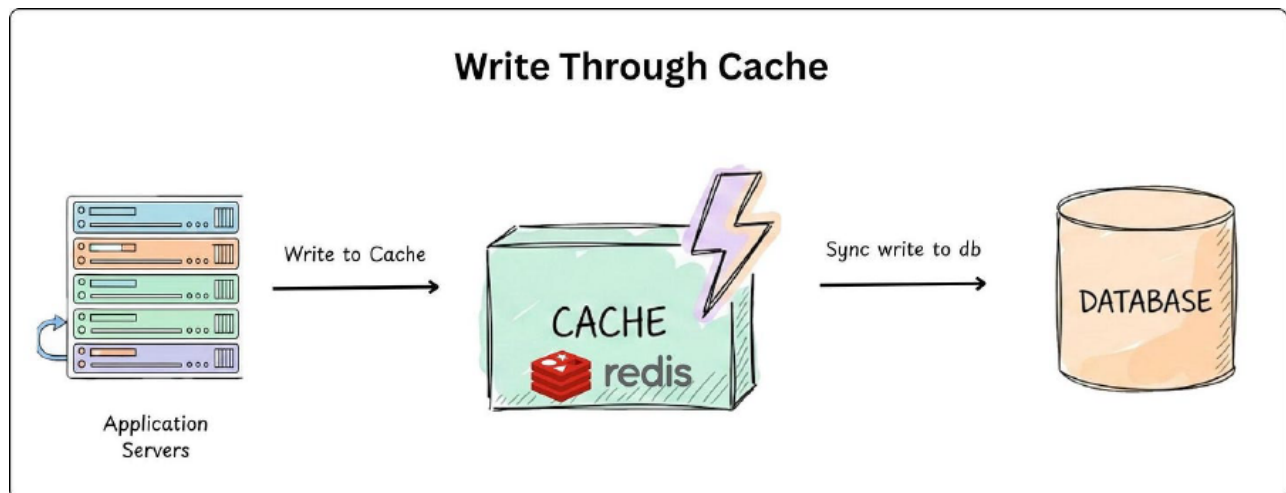
TRADE-OFFS

Risk of stale data if the DB is updated without cache invalidation.

BEST FOR:

High read-to-write ratios, such as e-commerce product data.

Write Through



THE FLOW

Every write operation is executed on both the cache and the DB at the same time.

LOGIC & OWNERSHIP

Synchronous process ensuring both layers update as one operation.

BENEFITS

Strong data consistency between cache and DB.

Future reads benefit from low latency as data is already cached.

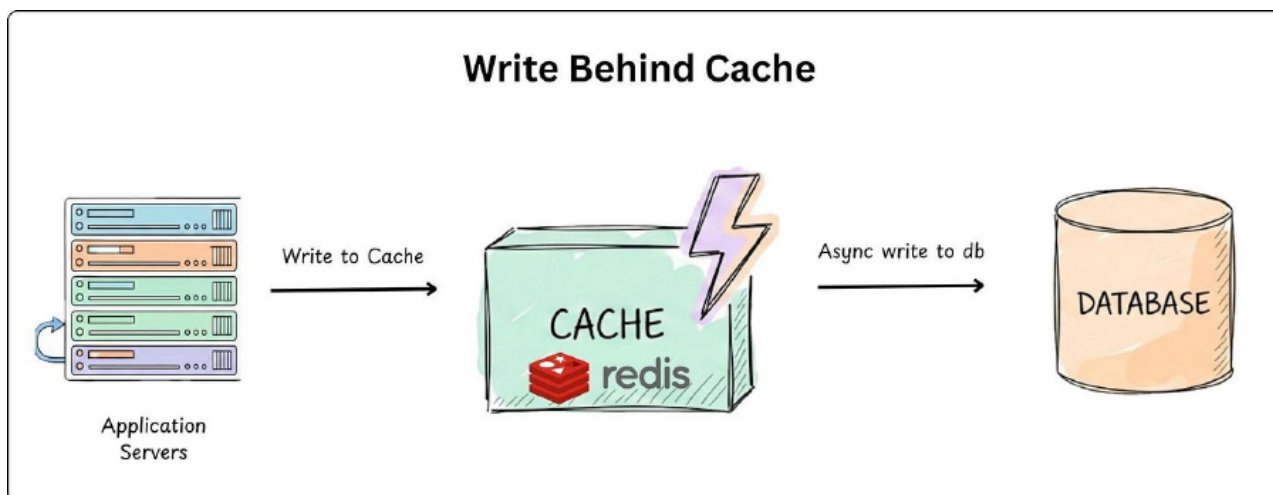
TRADE-OFFS

Higher write latency due to the overhead of two simultaneous writes.

BEST FOR:

Consistency-critical systems like financial or transaction apps.

Write Behind / Write Back



THE FLOW

Data is written to the cache first.

The database is updated asynchronously at a later time.

LOGIC & OWNERSHIP

Focuses on minimizing write latency by deferring database updates. Cache acts as the primary storage during the write process.

BENEFITS

Significantly reduces write latency.

High performance for frequent or batched write operations.

TRADE-OFFS

Risk of data loss if the cache fails before the data syncs to the DB.

BEST FOR:

Write-heavy scenarios like real-time analytics or “likes”.

Scaling a Cache

As the system and the traffic starts getting higher, then a single node won't be able to sustain this right as the CPU/RAM runs out of memory, so that's when we start to think about scale.

Cache is similar to a DB, so scaling also is similar to a DB.

Vertical Scaling

Horizontal Scaling - Either Read Replicas of cache instances (mostly for read heavy traffic), or Sharding the data (in case there are more writes in the system). You can refer this to read about database scaling strategies

Once we move to Horizontal Scaling of Caches, now the data is stored across multiple nodes, so it becomes a distributed cache.

When we scale a cache horizontally, we use a hash function to decide which server stores which key (e.g., $\text{server} = \text{hash}(\text{key}) \% \text{total_servers}$). Exactly like how we did in databases. And if you add or remove a server, the `total_servers` count changes, which causes almost all keys to re-hash to different servers. So we use consistent hashing here as well. You can read more about consistent hashing here -> [Consistent Hashing article](#)

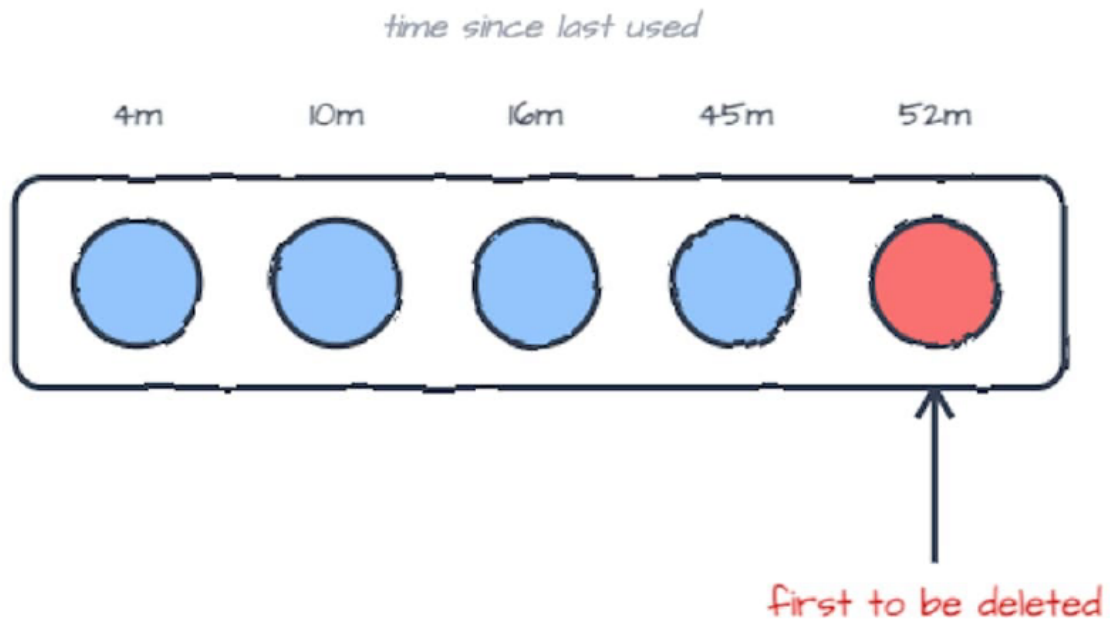
Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Cache Eviction Policies

We learnt that caches have limited memory. So when the cache gets full, we need a way to remove some existing entries to make space for new ones to keep the most useful (hot) data and evicting the least useful (cold) data.

There are multiple cache eviction policies, and i have summarized the most common ones below in an image

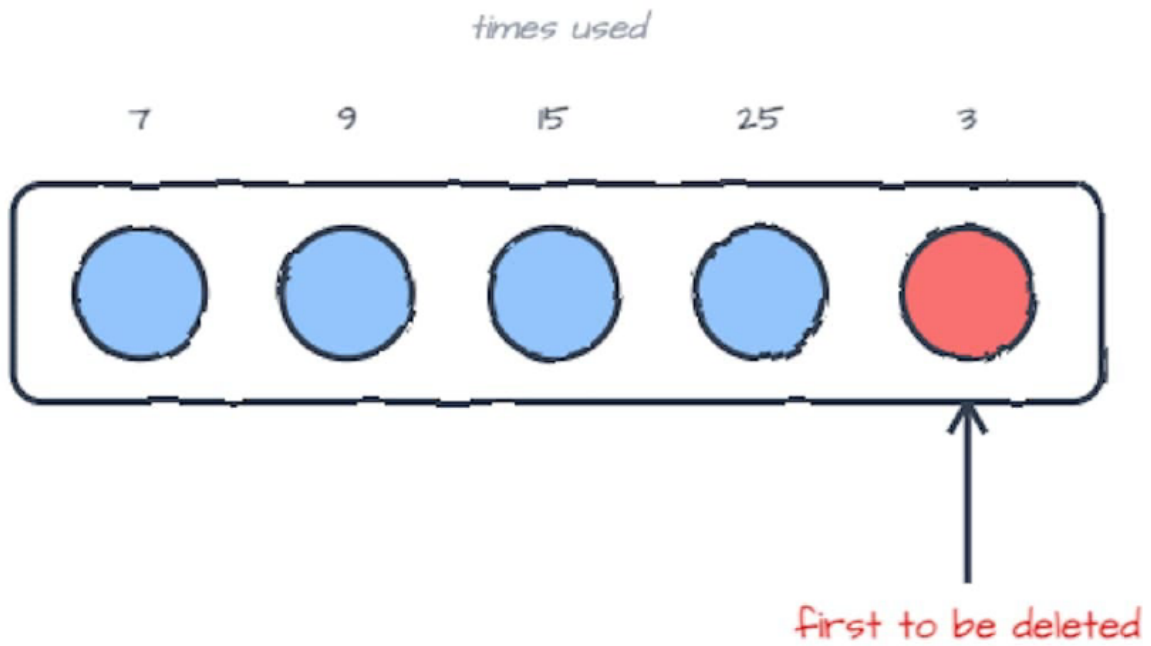
Least Recently Used (LRU)



LRU evicts the item that hasn't been used for the longest time.

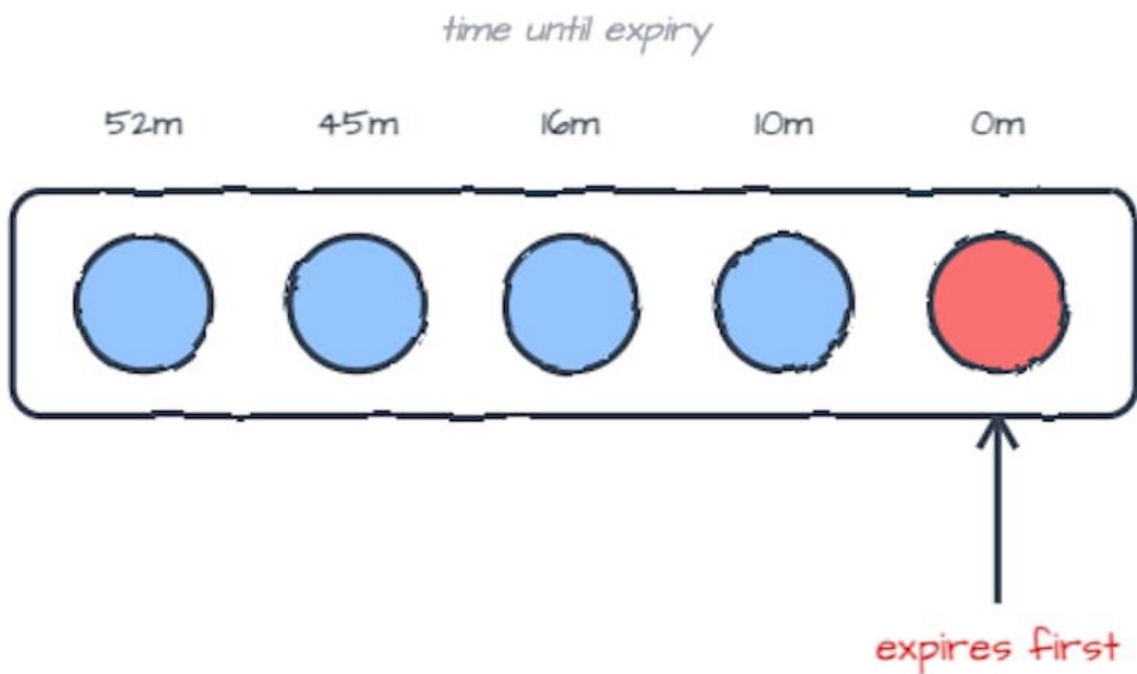
if you haven't accessed an item in a while, it's less likely to be accessed again soon.

Least Frequently Used (LFU)



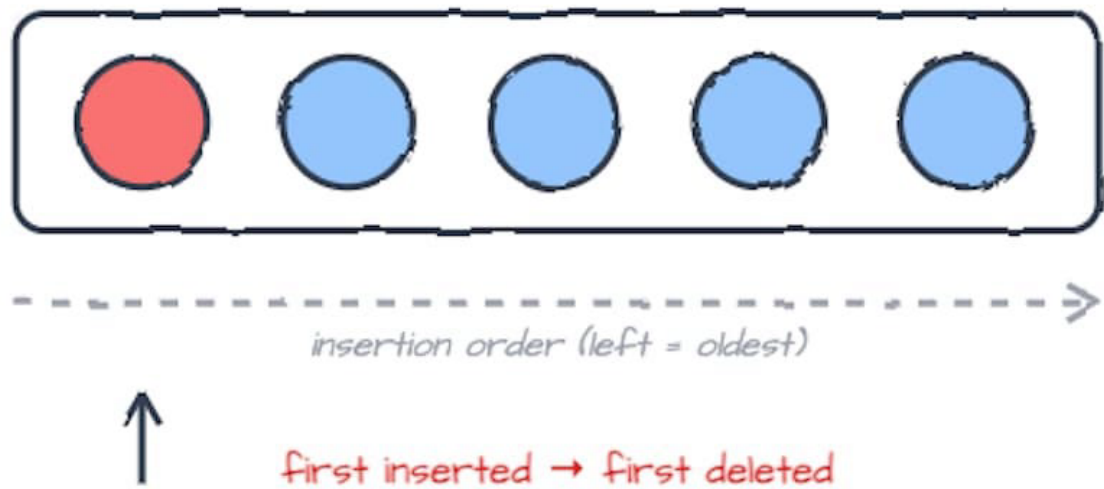
LFU evicts the item with the lowest access count.
if something is rarely used, it's a good candidate to remove.

Time To Live (TTL)



TTL evicts items after a fixed time window or expiry.
data becomes stale after a while, so it should expire automatically.

First In First Out (FIFO)



FIFO evicts the oldest item inserted into the cache.

what came in first gets removed first, regardless of usage.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Measuring Cache Effectiveness

After implementing such a robust system, it is important to test and measure the effectiveness of the system. This can be done a few ways :

METRIC	WHAT IT MEASURES	WHY IT MATTERS	IDEAL STATE
Cache Hit Rate	% of requests served by cache.	Shows if you are actually saving the DB.	High (80-99%). Low means wrong data is cached.
Eviction Rate	% of items removed because cache is full.	Signals if your cache size is too small.	Low. High means "Thrashing" (deleting useful data).
Expiration Rate	% of items removed via TTL.	Signals if your TTL is too short for the data type.	Moderate. Matches data volatility.
Consistency	Gap between DB and Cache.	Prevents users from seeing "stale" or old info.	Strong. Use "Write-through" for sensitive data.
AMAT	Average latency of data access.	Proves the cache is actually making the app faster.	Minimal. Should be much lower than DB latency.

Well while discussing about the TTL based cache eviction policy, you might have got this question in mind as to what's an ideal TTL to set in any system. Well, there is no "magic number" that works for every system, the ideal TTL is always a trade-off between Data Freshness (low staleness) and System Performance (high hit rate).

The more volatile the data, the shorter the TTL must be to ensure the user doesn't see stale data.

TTL Strategy by Volatility

DATA VOLATILITY	STRATEGY	EXAMPLE	IMPACT ON HIT RATE
High	Short TTL (Seconds)	Stock prices, Live Scores	Lower (Requires frequent refreshes).
Medium	Medium TTL (Minutes)	User profile, Session data	Balanced (Safe compromise).
Low	Long TTL (Hours/Days)	Product descriptions, Configs	Highest (Data stays in cache longer).

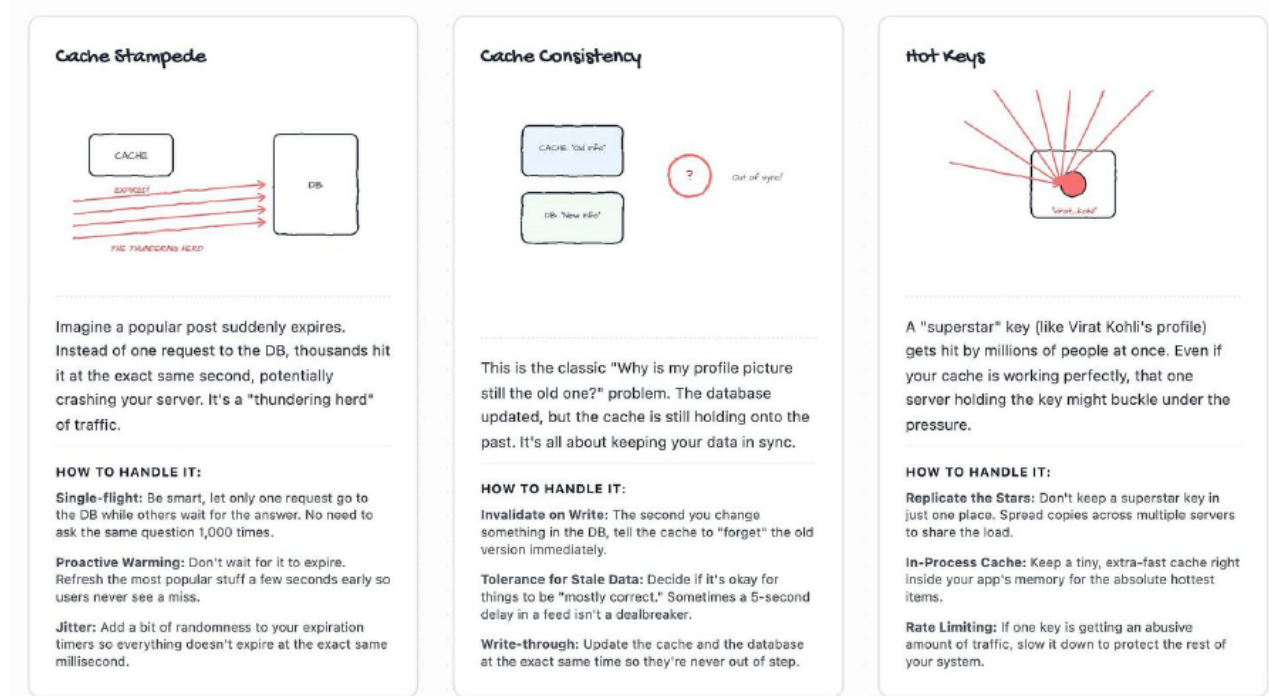
Common Edge Cases to handle in Real Systems

Cache Stampede

Cache Consistency

Hot Keys

Here's a wonderful gist i created; this image depicts these edge cases and how they show up in production traffic patterns.



Finally, hope this writing helped you in some ways to understand caching in real-world systems. Would like to end this article with a famous quote by Phil karlton

There are only two hard things in Computer Science: cache invalidation and naming things.

Hope you liked this article, and of you did then please do leave a ??, and subscribe to my newsletter, and share it with your friends on social media.

Share

A Simple Proof of the CAP Theorem

2026-01-03 · CAP in one scenario (no fluff)

Source: <https://pradyumnachippigiri.substack.com/p/a-simple-proof-of-the-cap-theorem>

Appreciate you reading this newsletter, I am writing as I learn too. I hope the content and resources are helping you level up as an engineer. If you want me to write about something specific, just message me or comment on the post.

Note: This article on was originally published on my blog - <https://pradyumnachippigiri.dev/blogs/cap-theorm>

Here's what I learned this week

2-random cache eviction

Ways to load images faster on web pages

Let's start by understanding the basic definition of CAP, and then lets delve deep to understand the what, why and how.



CAP Theorem states that in a distributed system, you can have only 2 out of the 3 of the following properties:

Consistency

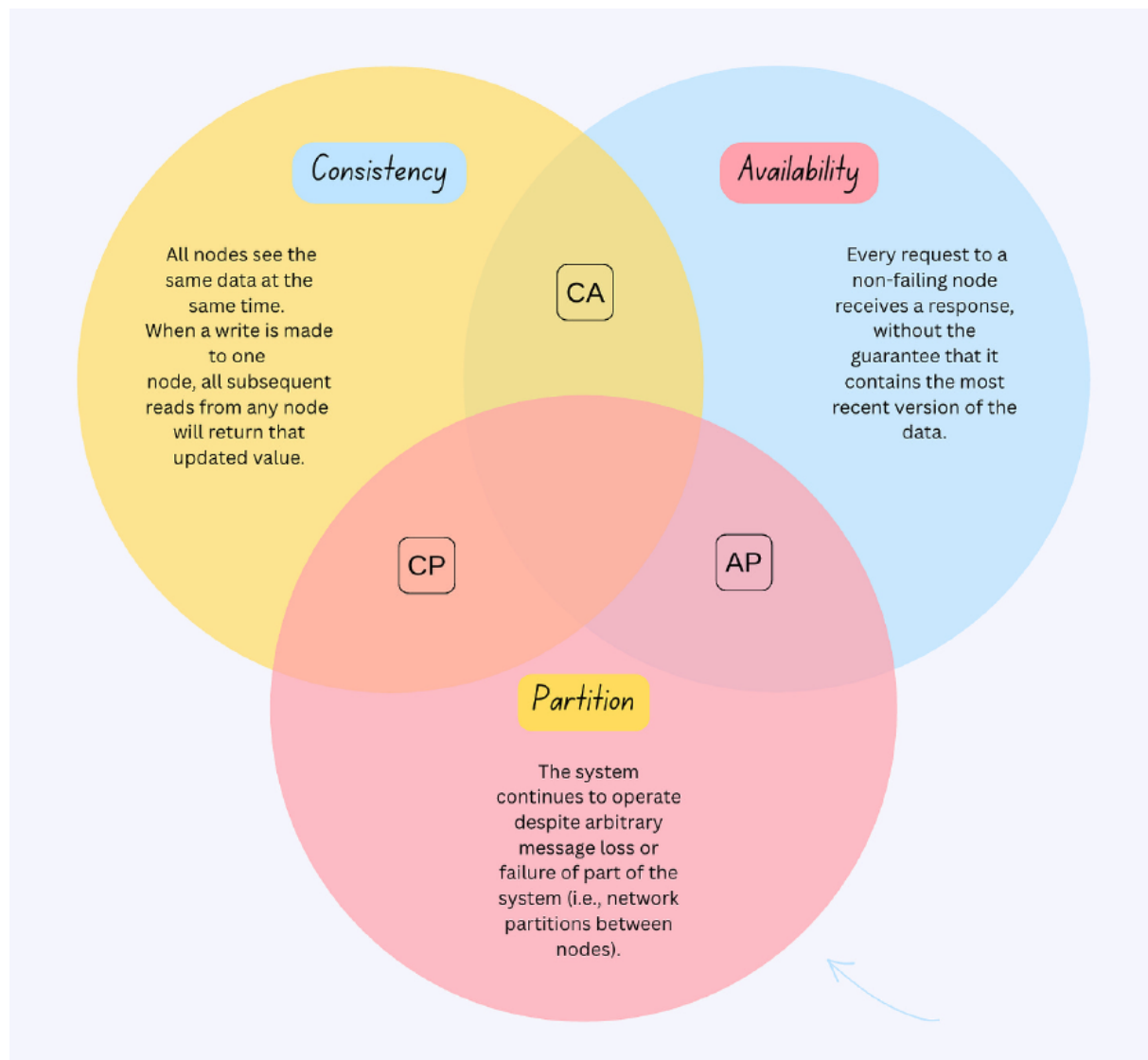
Every read receives the most recent write or an error. Consistency means that all clients see the same data at the same time, no matter which node they connect to. For this to happen, whenever data is written to one node, it must be instantly forwarded or replicated to all the other nodes in the system before the write is deemed succesfull. Consistency as defined in the CAP theorem is quite different from the consistency guaranteed in ACID database transactions.[]

Availability

Every request received by a non-failing node in the system must result in a response, without the guarantee that it contains the most recent version of the data.

Partition Tolerance

The system continues to operate despite arbitrary message loss or failure of part of the system (i.e., network partitions between nodes).



In the real world, network failures are inevitable. Cables are cut, routers fail, and packets are dropped. Therefore, Partition Tolerance (P) is a requirement, not a choice.

When a partition occurs, we are left with a binary choice: Consistency (CP) or Availability (AP).

To be honest, i just picked the above definitions from Wikipedia and pasted them here, as they have stated it beautifully.

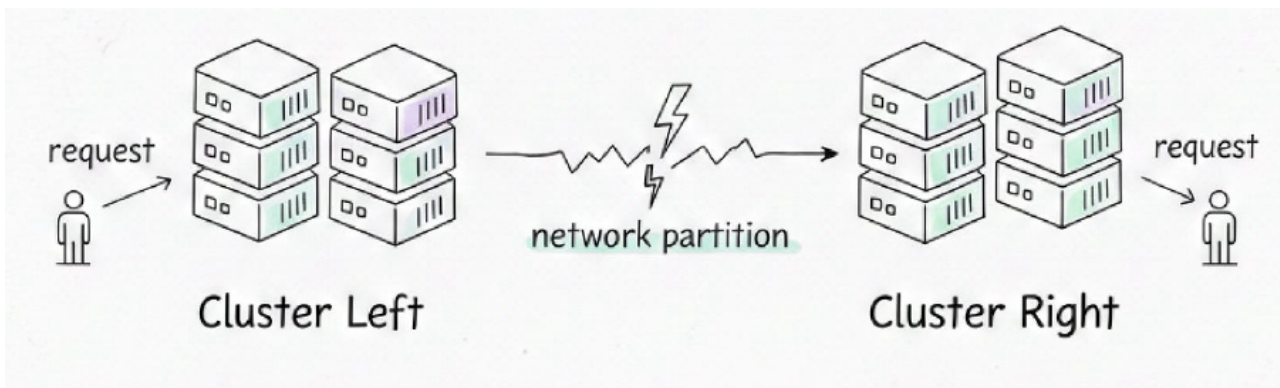
So now from the above definitions, the first question that may arise to you is :

Why cant there be a system with both consistency and availability ?

I thought that the CAP theorem was like this complicated topic in distributed systems theory.

It turns out that (even though it is a somewhat useful thing!) it is a really simple result to prove. Basically:

To understand why we can't have all three, let's look at a practical example



Imagine a distributed system with two clusters located in different regions: Asia(the cluster on the left) and the USA(the cluster on the right). Under normal conditions, they sync perfectly. But then, a submarine cable is damaged, and the two regions lose connection. They are now “partitioned.”

A user in Asia updates their profile. Since the Asia cluster cannot talk to the USA cluster, we have to make a design choice:

The CP Choice (Consistency): We want the USA user to see the updated data. But because the network is broken, we can't sync the update. To maintain consistency, we must return an error to the USA user (decreasing Availability). We'd rather show a “Service Unavailable” message than show the wrong data.

The AP Choice (Availability): We want the system to have zero downtime. Even though the clusters can't talk, the USA cluster still responds to the user. However, the user sees the old, stale data because the Asia update hasn't arrived yet. The system is available, but the data is inconsistent.

Which one should you choose ?

The choice totally depends on the usecase of the application.

CP based Applications

Ticket booking system (say a person booked a ticket, and partition happens, and if the pther person is able to see old data then even he can book the ticket, which is catastrophic, hence it has to be strongly consistent)

Financial banking Banking Applications.

These applications need strong consitency.

AP based Applications

Social Media App (if a person updates his name on his profile, and if you dont see it immediately (the correct data) thats okay, no one cares even if you sees stale data for sometime (but if the application goes down then thats not acceptable.))

Business review service etc.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Tools or types od DB for each usecase ?

CP based Applications

These kind of applications that demand strong consistency the db which satisfy these are :

Postgres

RDBMS

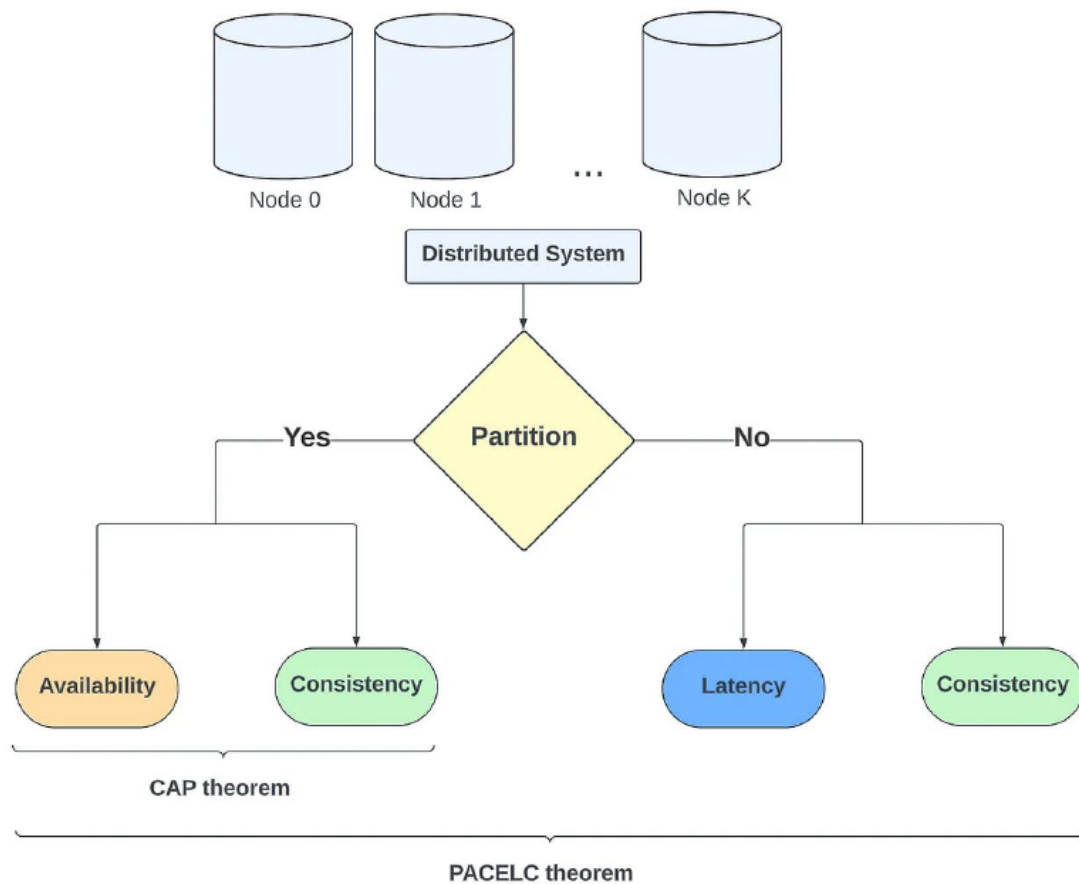
AP based Applications

these kind of applications which demand availability and where eventual consistency is okay then we use :

DynamoDB

Cassandra

Extended version of CAP



CAP theorem vs PACELC theorem

PACELC is another theorem, an extension of the CAP theorem. Wikipedia states that in the case of network partitioning (P) in a distributed computer system, one has to choose between availability (A) and consistency (C) (as per the CAP theorem), but else (E), even when the system is running normally in the absence of partitions, one has to choose between latency (L) and loss of consistency (C).

Hope you got a fair understanding of the CAP theorem, if you liked it please do subscribe and consider sharing it with your friends on social media if you found this helpful.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Easy Explanation of Consistent Hashing

2025-12-27 · Deep dive into consistent hashing, and why knowing about this is important

Source: <https://pradyumnachippigiri.substack.com/p/easy-explanation-of-consistent-hashing>

Appreciate you reading this newsletter, I hope the content and resources are helping you level up as an engineer. If you want me to write about something specific, just message me or comment on the post.

Note: This article on was originally published on my blog - <https://pradyumnachippigiri.dev/blogs/consistent-hashing>

If you find this helpful and interesting,
share it on HackerNews and other social media platforms.



Before we understand what Consistent Hashing is, let's understand what the problem was in traditional hashing based designs.

But firstly what is Hashing ?



Hashing

Hashing is a way to turn any input (string, number, object, file...) into a fixed-size value called a hash (or hash code) using a hash function.

Why is it useful ?

Fast Lookup: Hash tables/maps use hashes to find items in $\sim O(1)$ average time.

Partitioning: You can hash a key to decide which bucket/server should store it.

How do you decide if a function is a good hash function ?

It should be deterministic meaning: Same input should generate the Same Hash (always).

Uniform Distribution: Spreads the data evenly. (to avoid hot spots)

The Modulo %N Approach

In a distributed system, we need a way to map an incoming request (with a key like `user_id`) to one of our N servers.

The simplest way to do this is using the Modulo operator (%).

How it works:

We take the hash of the key: `hash("user_123") = 145892`.

We perform modulo N (where N is the number of servers): $145892 \% 3$.

The result is the index of the server (e.g., 0, 1, or 2). This is great because it's simple, fast, and distributes keys fairly evenly across your servers. As long as N stays the same, everything works perfectly.

Example :

Say we have a user called "user_123" and we want his request to always hit a particular database. But this question may arise as to why do we even need the user to hit the same server/DB ?

Because data and cache are usually partitioned, and sending the same user to different servers causes misses,

extra network calls, and higher latency. For example say:

DB shard 0 -> users 1–1M

DB shard 1 -> users 1M–2M

DB shard 2 -> users 2M–3M

If user_42's data lives in shard 2, then: every request for user_42 must go to shard 2, sending that request to shard 0 or 1 means the data is simply not there. You can read more about sharding and its different strategies here : [Sharding](#)

So consistent routing by user key is required for correctness especially for stateful systems like Database. We may not need it for our backend services because they are stateless, and any server can handle any request. There is no requirement that the same user has to hit the same server.

But the good practise is that we send the same user to the same server even for backend services and here's why :

In-Memory Session State:

If your server stores session data in RAM (not in Redis), you want the user to return to the same server. (Note: InMemory session states were in legacy systems, but as per modern best practises, it is always suggested to moving sessions to a distributed store like Redis)

This is called sticky sessions. (hash by cookie/userId)

Cache Locality:

If Server 1 already has user 42's profile cached in memory, hitting Server 1 again is fast.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Which Parts of a system does Hashing happen ?

Hashing can happen at multiple layers like :

Load Balancer -> App Servers

App Servers -> Cache Cluster (v. common as hashing chooses which cache node stores a cache key)

App Servers -> DB Shards (v. common as hashing chooses which particular shard a users data lies in)

Message Queues/streaming -> Kafka partitions: $\text{partition} = \text{hash}(\text{key}) \% \text{numPartitions}$

Commons Hashes used in the Industry

In real systems, common choices:

MurmurHash

xxHash

CRC32

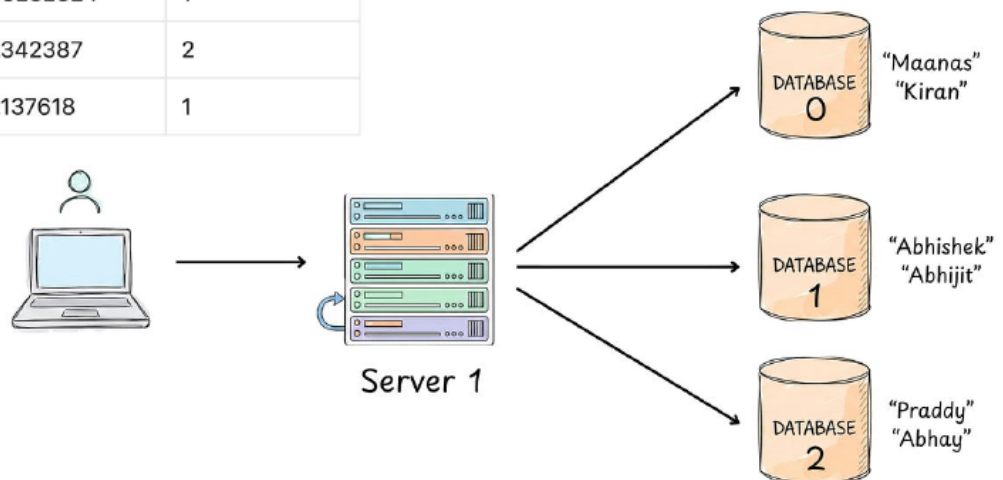
sometimes SHA-1 / SHA-256 (more expensive, but stable)

For learning, you don't need the exact function. Assume you have a black box called `hash()` that always gives you the same number for the same key.

The Rehashing Problem

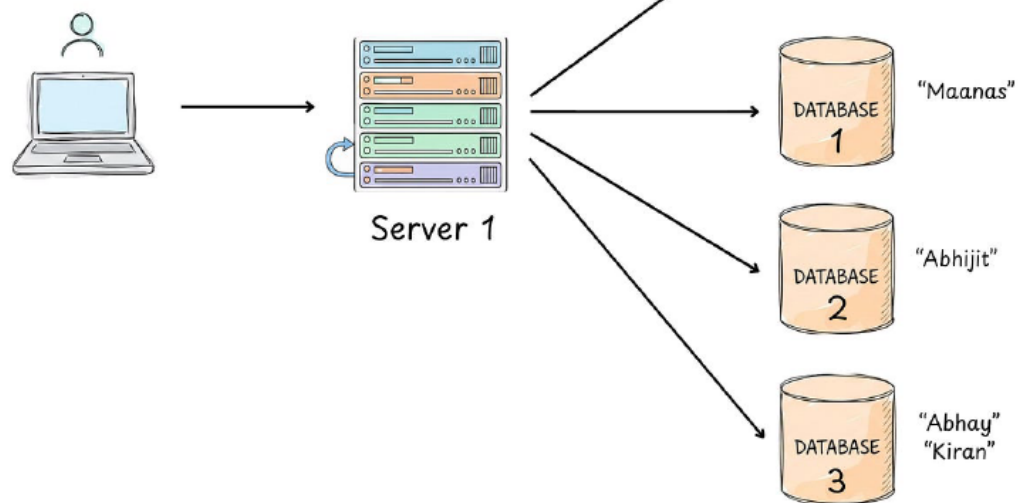
The approach that we talked about worked great at the start. Say initially we had a setup like this :

Key	Hash(key)	Hash(key) % N
Praddy	1123232	2
Kiran	87327	0
Maanas	562569	0
Abhishek	43232824	1
Abhay	2342387	2
Abhijit	2137618	1



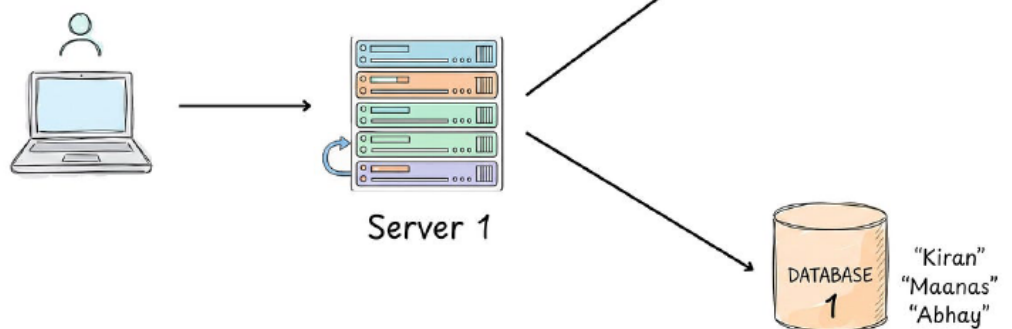
But say our site continues to grow now, so we would need more servers, so in that case we would change the modulo in that function right, meaning if we initially had 3 servers then N was 3 and now after scaling N becomes 4. So this means, now all the data needs to be re-distributed.

Key	Hash(key)	Hash(key) % N
Praddy	1123232	0
Kiran	87327	3
Maanas	562569	1
Abhishek	43232824	0
Abhay	2342387	3
Abhijit	2137618	2



Similarly, when we want to downgrade our system, so the $N = 2$ now, in such cases too we would need to redistribute the data across nodes.

Key	Hash(key)	Hash(key) % N
Praddy	1123232	0
Kiran	87327	1
Maanas	562569	1
Abhishek	43232824	0
Abhay	2342387	1
Abhijit	2137618	0



As you can observe from the above two diagrams, there is a main problem with the normal hashing which relies on the number of servers for the modulo value.

Every time there is a datastore node added/removed, we need to consider rehashing all the keys present across all of our datastore nodes. This is a super expensive operation in terms of shuffling and redistributing the whole data.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

The Solution : Consistent Hashing

Consistent hashing addresses the issue that we discussed above by ensuring that only a small subset of keys need to be reassigned when nodes are added or removed. It performs really well when operated in dynamic environments, where the system scales up and down frequently

How Consistent Hashing Works ?

We imagine a ring like structure where we place our datastore nodes and the keys. And we come up with a mechanism where only a small subset of data has to be changed and in this approach we try to :

Keep the hash function independent of the number of storage nodes

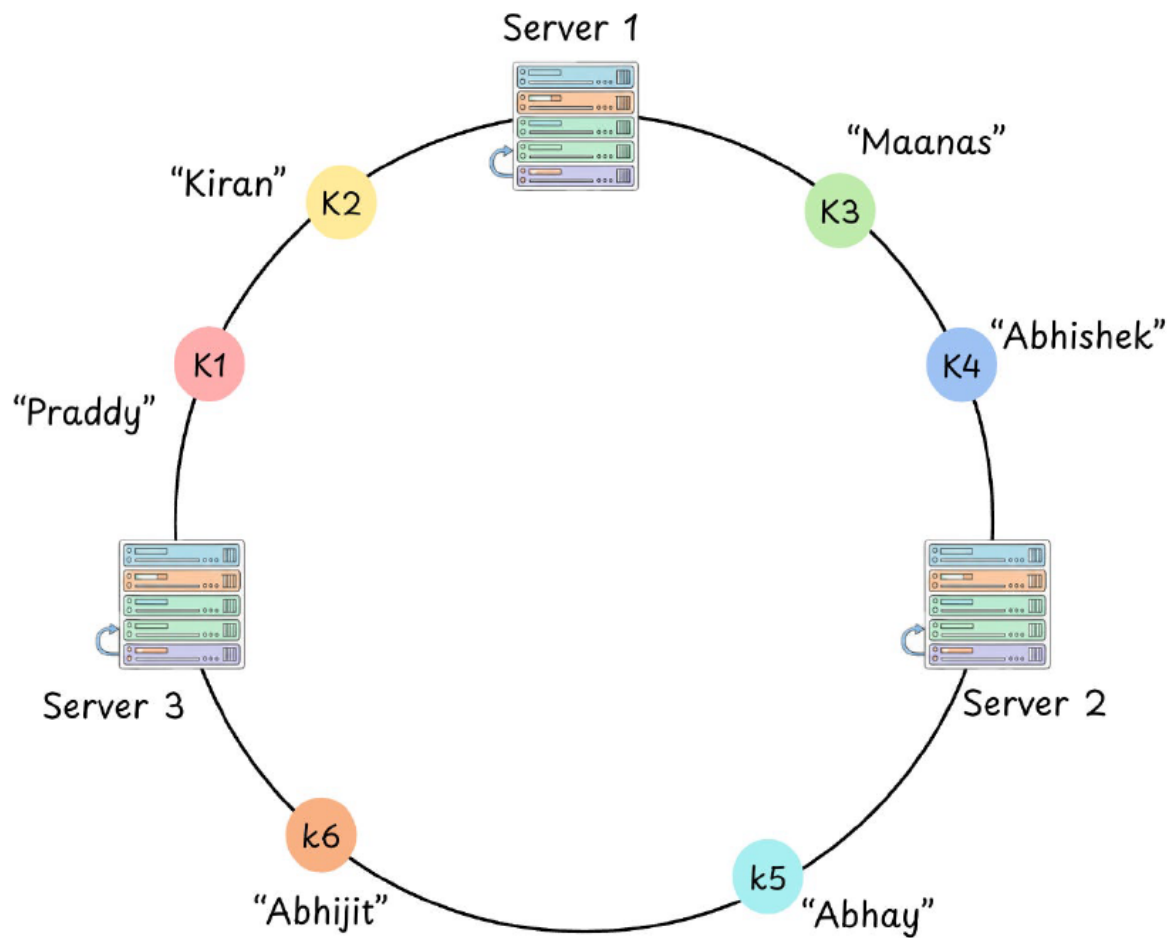
The Hash Space

We treat the output of our hash function as a fixed circular range. For example, if we use a 32-bit hash, the range goes from 0 to $2^{32} - 1$. Imagine this range wrapped around so that the maximum value connects back to zero .

Placing Nodes and Keys

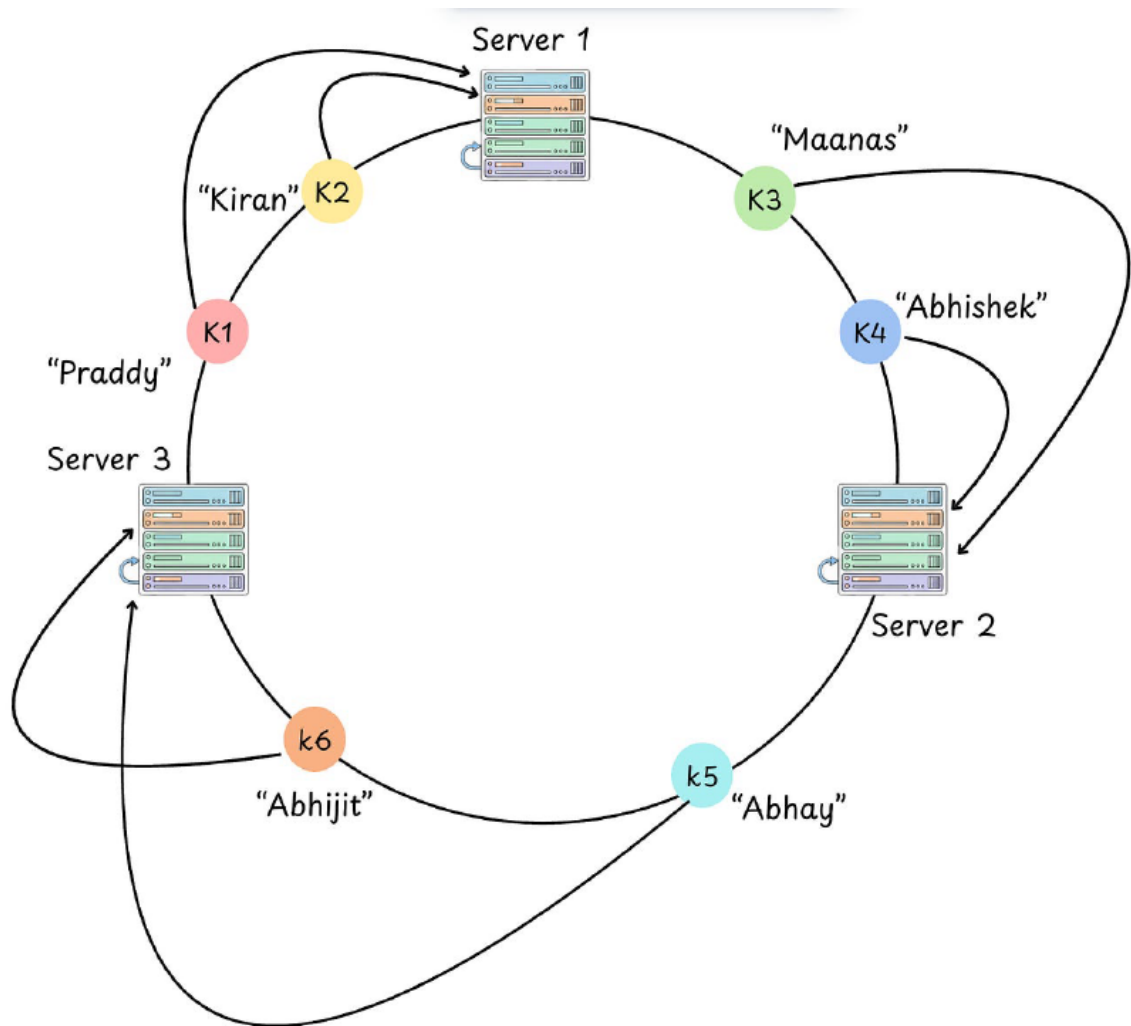
Nodes: We hash the node's identifier (like its IP address or name) to get a position on the ring.

Keys: We hash the data key to get its position on the same ring.



The Lookup Mechanism

To determine which node "owns" a key, we locate the key on the ring and move clockwise until we encounter the first node. That node is responsible for storing that specific piece of data.



Segmenting the Ring:

When you plot M nodes on the ring, you are effectively creating M partitions. Because a good cryptographic hash function (like SHA-1 or MurmurHash) produces a uniform distribution.

Total Hash Space: Let's call the full range S .

Average Segment Size: S/M

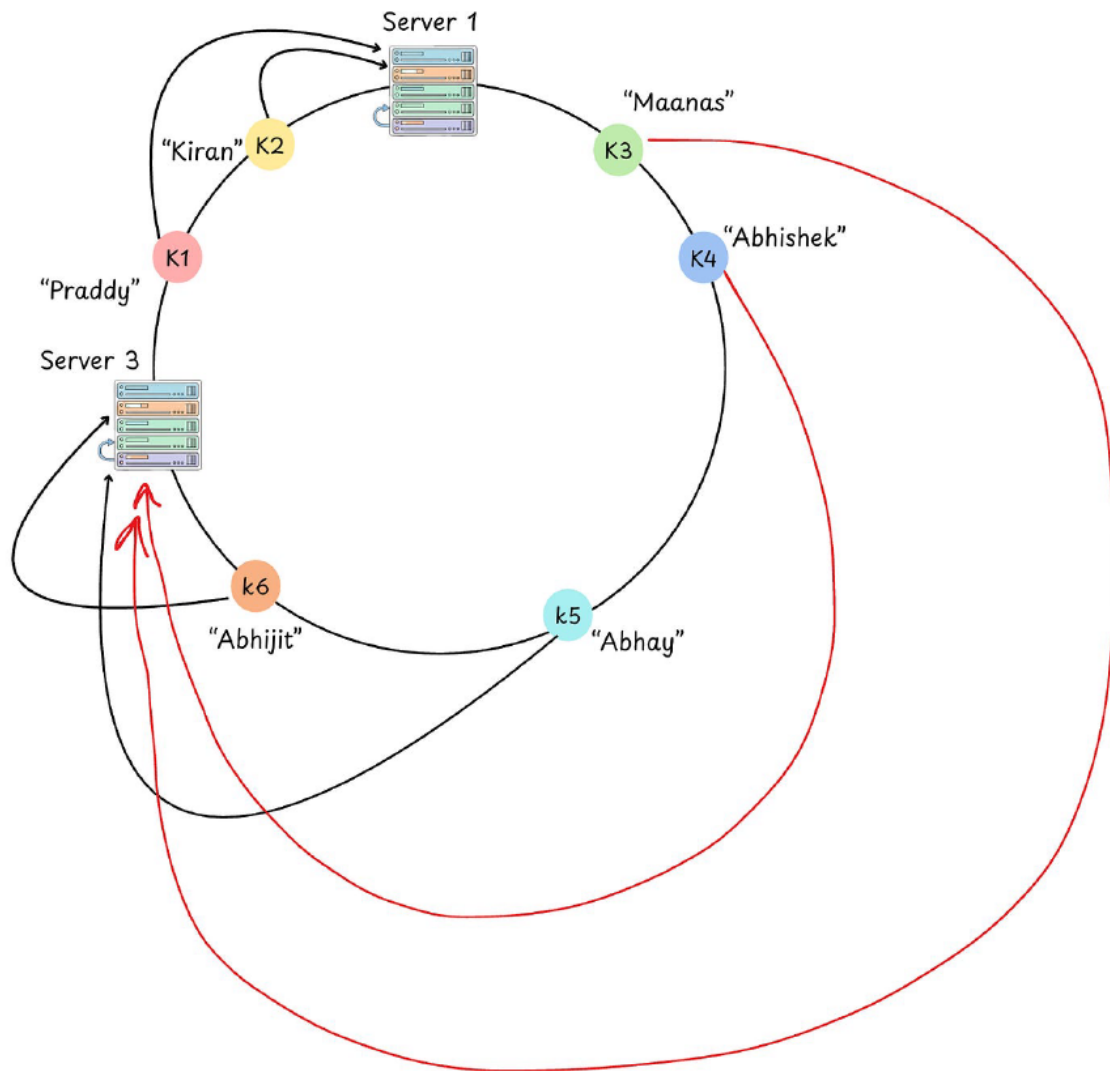
Expected Keys per Segment: Since keys are also uniformly distributed, each segment captures roughly N/M keys.

In a traditional hash table, if a server is removed, the entire index $\%M$ breaks. In consistent hashing, the "damage" is strictly bounded.

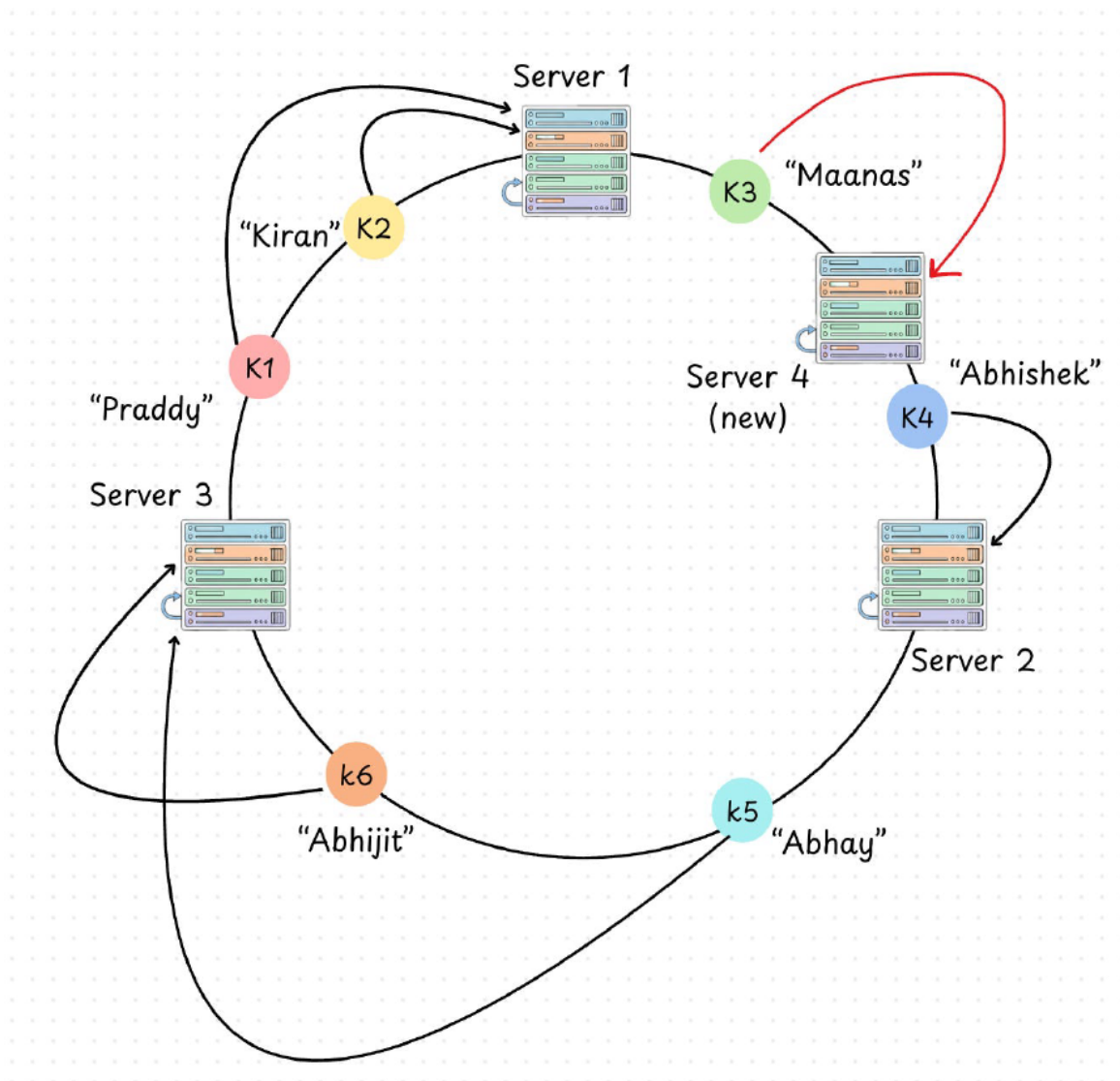
If Server 2 fails:

The Affected Keys: Only the keys sitting in the segment between Server 1 and Server 2 are "lost." These represent roughly N/M of your total data.

The Re-assignment: Following the clockwise rule, these keys are now mapped to Server 3 (the next available node). System Stability: The other $M-1$ nodes continue to serve their own N/M keys without any interruption or data movement.



The same logic applies when adding a node. If you insert a new server node between Server 1 and Server 2. It "slices" Server Node 2's segment. It takes over roughly N/M keys from Node 2. Other nodes are completely unaware that a change even occurred.



Now there is another problem, that has to be dealt with and that is the Neighbor Problem

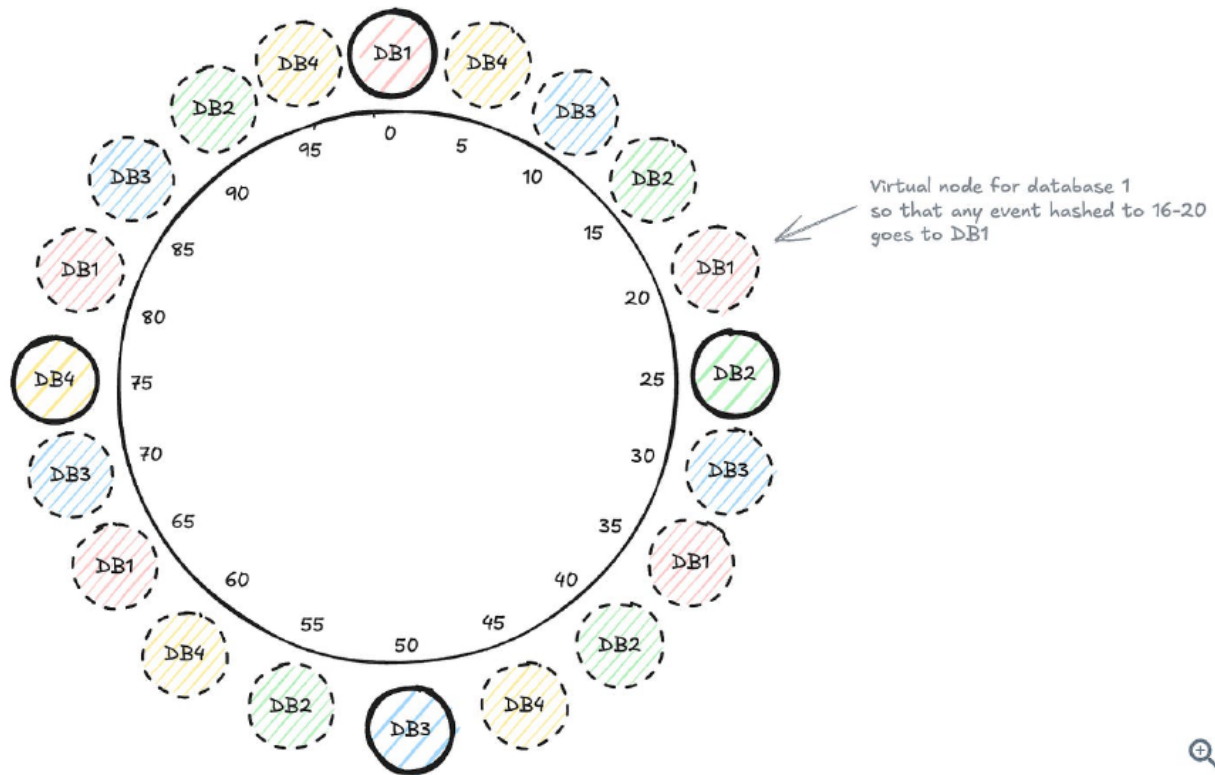
Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Virtual Nodes

While it's true that only N/M keys are rehashed, there is a catch: One single neighbor (the next node clockwise) inherits the entire load of the failed node. If Node 2 fails, Node 3 now has to handle its original load plus Node 2's load ($N/M + N/M$). In a high-traffic environment, this sudden 2x load on a single server can trigger a cascading failure, where Node 3 crashes from the weight, passing the combined load to Node 4, and so on. This is why Virtual Nodes are essential.

They ensure that when a physical node fails, its N/M keys are actually redistributed across multiple physical neighbors, rather than just one. Now this brings us to the topic of Virtual Copies.

Virtual nodes (VNodes) are a technique used in consistent hashing to improve load balancing and fault tolerance by distributing data more evenly across servers. Picking the image from Hello Interview



Hash Ring with Virtual Nodes

Now when Database 2 fails:

The events that were mapped to "DB2-vn1" will be redistributed to Database 1

The events that were mapped to "DB2-vn2" will go to Database 3

The events that were mapped to "DB2-vn3" will go to Database 4 And so on...

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Concurrency Controls : Pessimistic vs Optimistic

2026-03-22 · Understanding how systems prevent and handle conflicting transactions under the hood

Source: <https://pradyumnachippigiri.substack.com/p/concurrency-controls-pessimistic>

Appreciate you reading this newsletter, I am writing as I learn and explore these topics in depth. I hope the content and resources are helping you level up as an engineer. If you want me to write about something specific, just message me or comment on the post.

Note: This article on was originally published on my personal website -
<https://pradyumnachippigiri.dev/blogs/optimistic-locking-vs-pessimistic-locking>

Also, checkout all my Linkedin tech posts here -> <https://pradyumnachippigiri.dev/blogs/linkedin-posts>

Here's what I learned this week

Local SSD vs Network SSD

RLE Run Length Encoding

Pull vs Push CDN

In real systems, there might be multiple transactions that happen at the same time like :

Multiple users reading the same row

Multiple users updating the same row

Reads happening while writes are in progress

This leads to different types of conflicts:

Thanks for reading Evolving Engineer! Subscribe for free to receive new posts and support my work.

Read–Read

Read–Write

Write–Read

Write–Write

These concurrent interactions can cause anomalies like dirty reads, non-repeatable reads, phantom reads, Lost updates.

To control these concurrency anomalies databases provide isolation levels like Read Committed, Repeatable Read, and Serializable.

You can read about Concurrent Anomalies and Isolation Levels in depth in one of my previous article here :

But an important question is:

How are these isolation levels actually implemented under the hood?

The answer is: using different concurrency control mechanisms.

There are three major approaches:

Pessimistic Concurrency Control

Optimistic Concurrency Control

MVCC (We will talk about this in our next article)

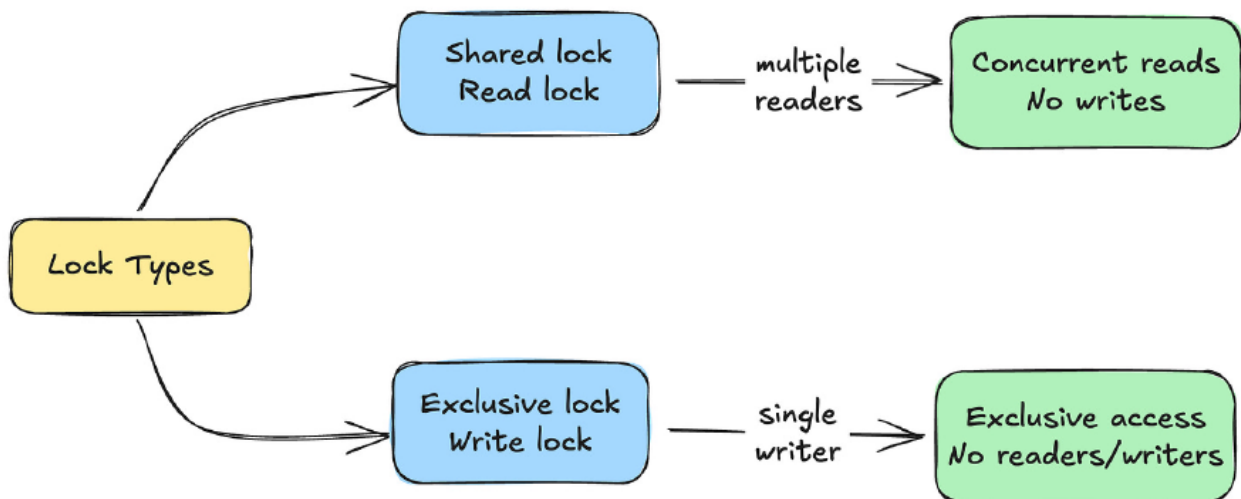
Optimistic locking or Pessimistic locking are just approaches for concurrency control, they're not specific keywords in the database query language sense.

When dealing with conflicts, we have two options:

we can try to avoid the conflict, and that's what Pessimistic Locking does.

Or, we allow transactions to proceed concurrently and detect conflicts at commit time, and that's what Optimistic Locking does.

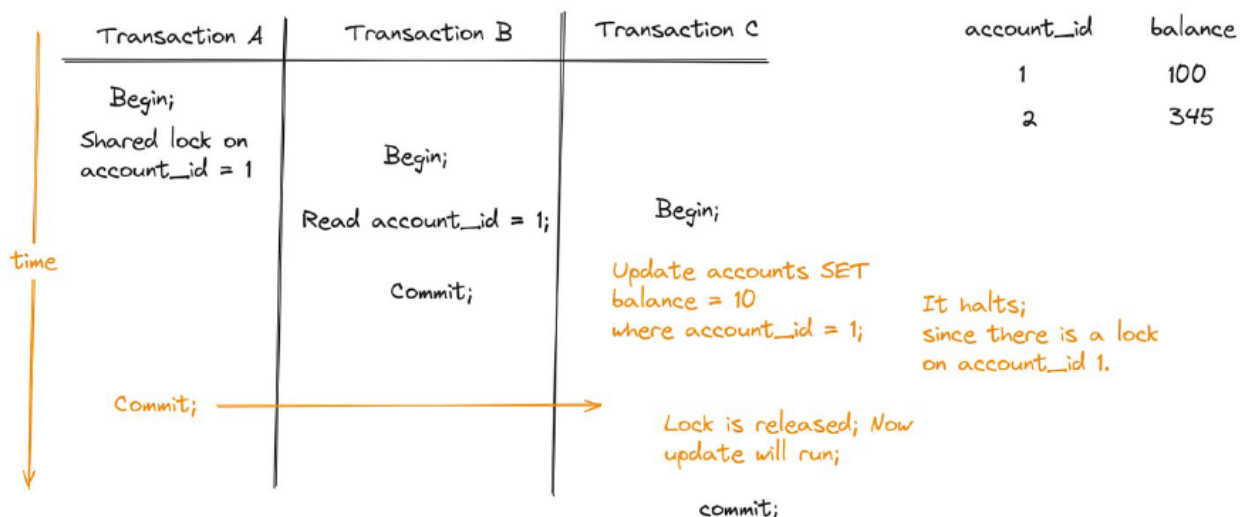
Lets understand the 2 basic locks before we understand concurrency control techniques.



Thanks for reading Evolving Engineer! Subscribe for free to receive new posts and support my work.

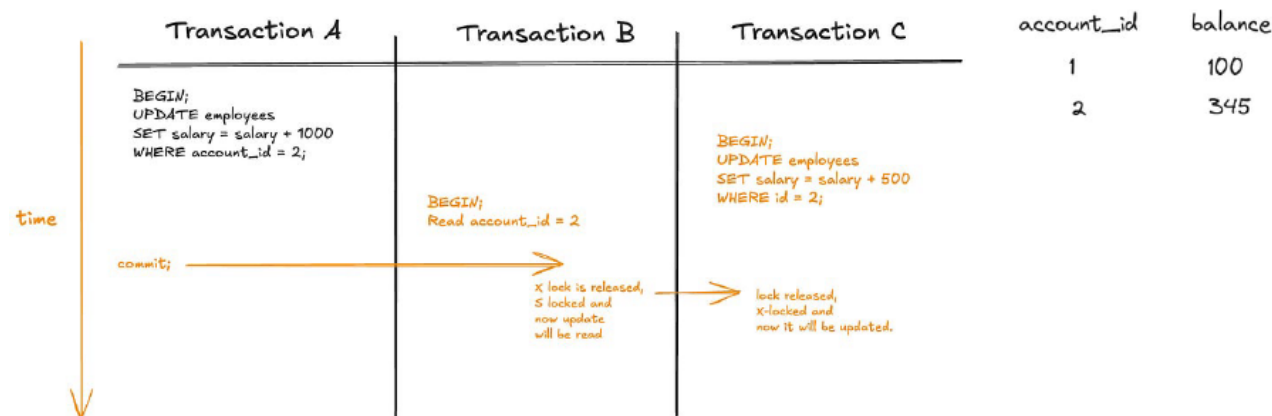
Shared Lock (S-Lock)

A shared lock is used when a transaction needs to read a resource (such as a database row or table) without modifying it. Multiple transactions can acquire a shared lock on the same resource at the same time. However, as long as one or more shared locks exist on a resource, no transaction can modify/update/write to that resource.



Exclusive Lock (X-Lock)

An exclusive lock is used when a transaction wants to change data. Only one transaction can hold this lock at a time, and while it is held, no other transaction can read or write that data.



Thanks for reading Evolving Engineer! Subscribe for free to receive new posts and support my work.

Pessimistic Concurrency Control

The literal meaning of pessimistic is “believing that bad things will happen”.

Pessimistic concurrency control assumes that conflicts between transactions are likely. So, before a transaction performs operations on data, it acquires locks to prevent other transactions from accessing the same data in a conflicting way.

In simple terms, it follows this idea:

since conflicts are likely, prevent them upfront instead of detecting them later.

If a transaction wants to read data, it may acquire a shared lock. If a transaction wants to modify data, it acquires an exclusive lock.

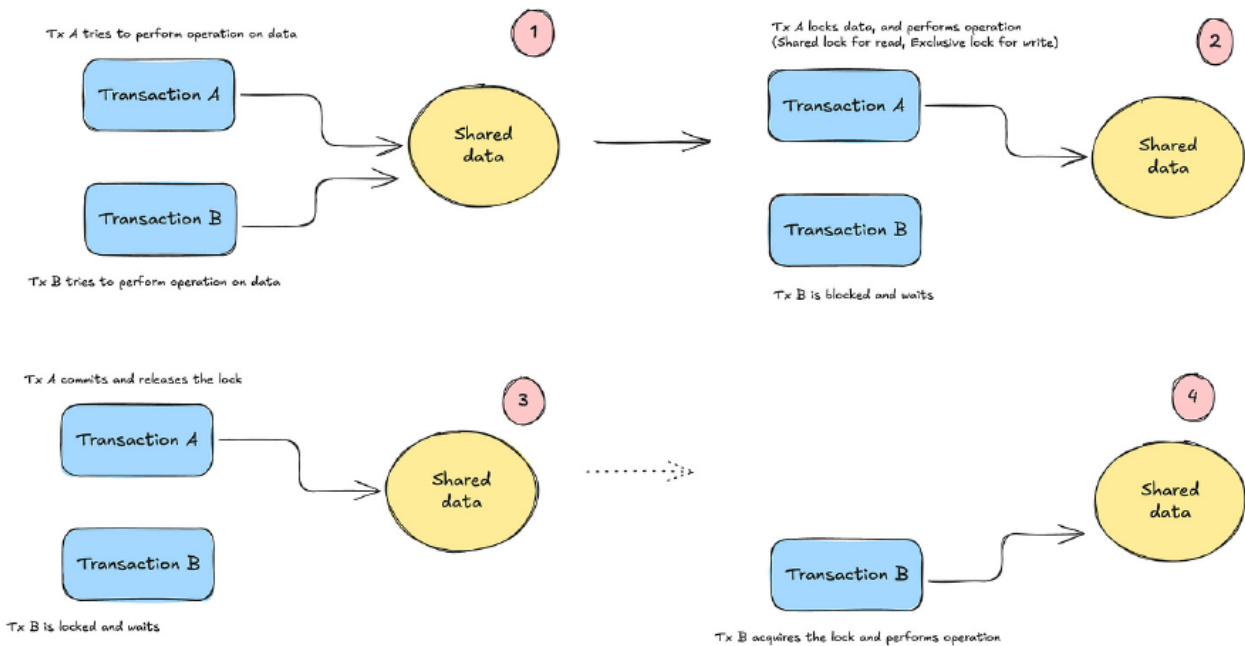
This means:

Multiple transactions can still read the same data at the same time using shared locks

But once a transaction acquires an exclusive lock on a resource, other transactions must wait before they can read or write that same resource

So pessimistic concurrency control avoids concurrency anomalies and other conflicting operations by making transactions wait until the current lock is released.

Example



Limitations and Challenges

Reduced Concurrency – Exclusive locks prevent other transactions from accessing the locked resource, which can lead to bottlenecks in highly concurrent systems.

Blocking Writes – Shared locks can delay write operations, potentially impacting performance in write-heavy systems.

Deadlocks - Example:

T1 locks row A, wants row B

T2 locks row B, wants row A

Now both wait forever unless database detects and breaks the deadlock.

Higher latency - Transactions will be spending time waiting for locks to be released. So response time can increase

Lock Management Overhead – Tracking, maintaining, and resolving locks adds internal overhead to the database.

Thanks for reading Evolving Engineer! Subscribe for free to receive new posts and support my work.

Optimistic Concurrency Control

The literal meaning of Optimistic is “hoping good things will happen”.

Optimistic concurrency control assumes that conflicts between transactions are rare. So instead of locking data before performing operations, it allows transactions to proceed freely and checks for conflicts only later, usually at commit time.

In this approach, transactions typically read the currently visible committed data without taking long-held locks upfront, perform their work, and then verify at commit time whether the data has changed in the meantime.

The optimistic lock is not implemented by the locking mechanism of the database management system. Instead, it is more on the application code level lock.

For example, a transaction may:

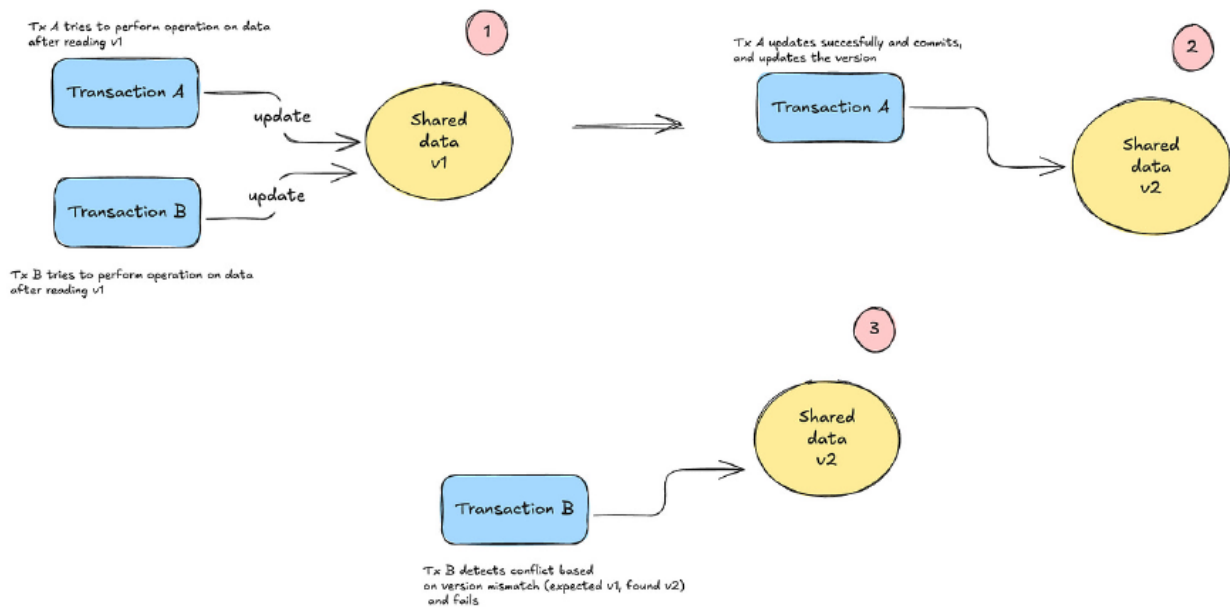
read a row along with its version

perform updates in memory

attempt to update the row only if the version has not changed

If the version has changed, it means another transaction has already modified the data, and the current transaction fails and must be retried.

Example



Limitations and Challenges

Late Conflict Detection – Conflicts are detected only at commit time, after the transaction has already done its work.

Wasted Work – Multiple transactions may perform the same operations, but only one succeeds while others fail.

Poor performance under high contention – frequent conflicts can lead to repeated failures and retries.

Additional Metadata Requirement – Requires versioning or timestamps to detect conflicts.

Retries Required – Failed transactions must be retried, increasing application complexity.

Conclusion

Each of these strategies has its own use cases.

Pessimistic concurrency control is preferred when conflicts are likely, when retrying failed work would be expensive, and when strong correctness and consistency are critical. It is a good fit for systems like banking, ticket booking, inventory management, and other high-contention workloads where it is better to make transactions wait rather than risk conflicting updates.

Optimistic concurrency control is preferred when conflicts are rare, contention is low, and occasional retries are acceptable. It works well for read-heavy systems, low-conflict business workflows, and application-level updates where blocking transactions upfront would unnecessarily reduce concurrency.

The right choice depends on the workload, the likelihood of conflicts, and whether waiting or retrying is more acceptable for the system.

In the next article, we will explore MVCC and see how modern databases avoid the reader-writer blocking seen in traditional pessimistic locking by maintaining multiple versions of data.

If you liked this article, please do like, subscribe, and drop a comment to spark a discussion, so we can all learn from each other. And if you think it'll help someone, share it with your friends or on social media.

Thanks for reading Evolving Engineer! Subscribe for free to receive new posts and support my work.

If you'd like 1:1 help (system design, backend, DSA, resume review etc.), you can book a session with me on Topmate

If you found this useful and want to support my work, you can also buy me a coffee ?
buymeacoffee.com/cpradyumnao

I also write short form content on a daily basis on X and LinkedIn. Please do follow there too..

Client-Side Proxy vs Server-Side Proxy

2026-01-25 · Understanding forward proxies and reverse proxies from first principles, plus how they differ from VPNs and firewalls

Source: <https://pradyumnachippigiri.substack.com/p/client-side-proxy-vs-server-side>

Appreciate you reading this newsletter, I am writing as I learn and explore these topics in depth. I hope the content and resources are helping you level up as an engineer. If you want me to write about something specific, just message me or comment on the post.

Note: This article on was originally published on my personal website - <https://pradyumnachippigiri.dev/blogs/forward-proxy-vs-reverse-proxy>

Here's what I learned this week

My experience with Amazon Q + Mermaid

DORA metrics

llms.txt endpoint

Let's start with a very simple analogy to understand the both proxy types, and then let's understand what they are, why they are used.

Simple Analogy for Client-Side Proxy



Imagine you are a very famous person, and you want to buy vegetables for cooking, but you don't want anyone to recognize you. So you send your personal assistant to the store to buy them for you. The store owner only sees the assistant, sells the vegetables, and has no idea who they're actually for. In other words, your assistant

is hiding your identity from the outside world.

Here:

You are the client

Your personal assistant is the forward proxy

The store / store owner is the internet website / server

So a forward proxy sits between the client and the internet, makes requests on behalf of the client, and helps hide the client's identity (plus it can apply rules like allow/block/log if needed).

Simple Analogy for Server-Side Proxy



When you go to a restaurant and want to order food, you don't walk into the kitchen and talk to the chefs directly, right? Instead, you place your order with the waiter/host. They check the menu, make sure the order is valid, and then pass it to the chef so it can be prepared.

In this scenario:

You are the client

The waiter/host is the reverse proxy (server-side proxy)

The chef/kitchen is the server.

So the reverse proxy's job is to sit in between and make sure the client doesn't interact with the server directly, it accepts the request, applies rules/filters if needed, and forwards it to the right server. The reverse proxy(the host/waiter) is protecting and managing the kitchen (server) from bad/too many/invalid requests.

Thanks for reading Evolving Engineer! Subscribe for free to receive new posts and support my work.

Hope you got a fair understanding with this analogy, but now let's dive in.

What is Proxy ?

Proxy is nothing but a machine that sits between 2 systems. A proxy terminates the incoming connection and initiates a new, separate connection to the destination.

It acts like an intermediate layer that can:

apply rules (allow/block),

route traffic,

hide details of the other side,

and make communication more controlled and predictable.

Common misconception is that it can be only between a client and a backend servers, but it can also be between 2 backend servers too.

Depending upon the kind of proxy that we are using in our system, it will have different applications:

Forward Proxy: This type of proxy is used to protect the clients accessing the websites via the internet.

Reverse Proxy: This type of proxy is used to guard the web servers from the clients trying to access websites via the internet.

Let's understand about each of them in depth.

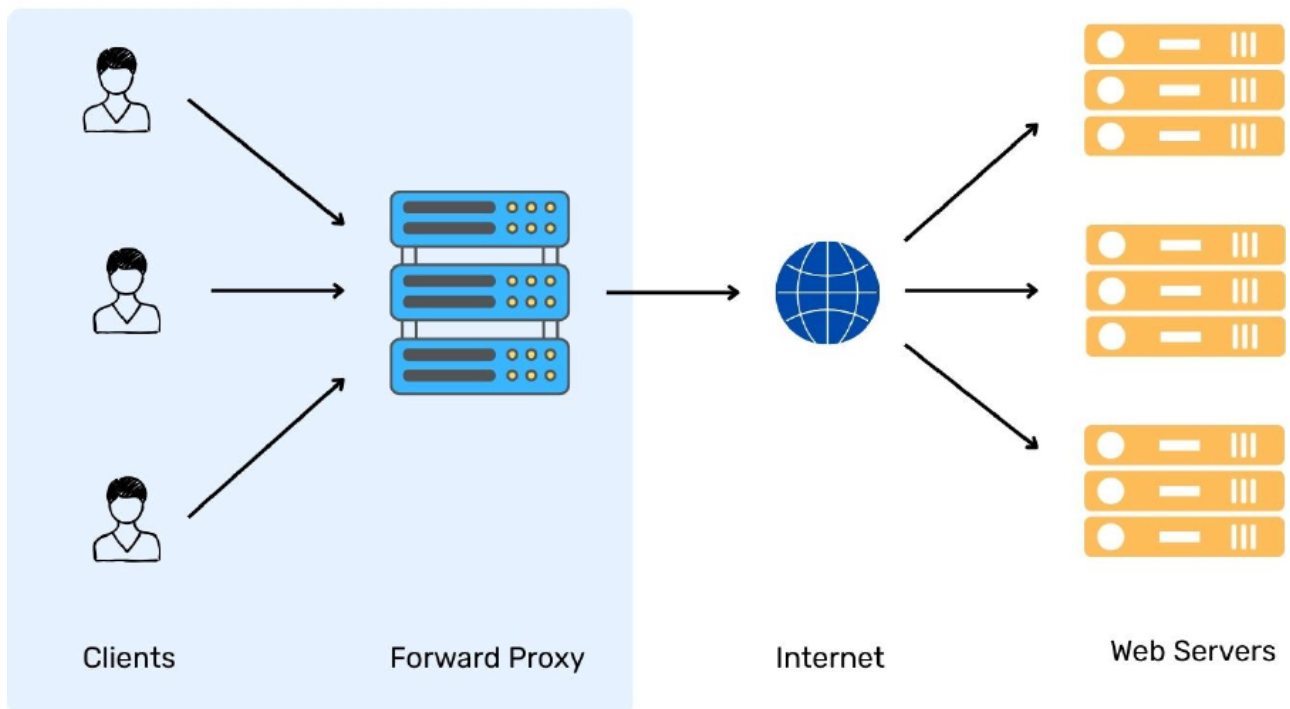
Forward Proxy (Client-Side Proxy)

Forward Proxy (also called a client-side proxy or internet-facing proxy) sits between the client and the internet (for example, between your browser and a website). So whenever your browser makes a request, it sends (in plain text without encryption) that request to the forward proxy first. The forward proxy can decide whether to allow it, block it, or route it to the correct destination server.

Then the destination server sends the response back to the forward proxy, and the forward proxy forwards that response back to your browser.

So the forward proxy's job is to act as a middleman for the client, it can hide the client's real identity (IP) from the internet and also control/log what the client is allowed to access.

Forward Proxy



Advantages of Forward Proxy

Content filtering and security : They can filter out unwanted or dangerous content. Proxies can block access to specific sites and protect the network by filtering out suspicious requests or malicious files before they reach the user.

Anonymity: A forward proxy hides the client's IP address from the destination server. Since the server only sees the proxy's IP, it makes it much harder to track individual user activity or target specific clients for an attack.

Caching : Forward proxies cache frequently requested resources. When another requests the same data, the proxy serves it directly from its cache instead of fetching it from the internet. This reduces bandwidth usage and speeds up load times.

Request Modification: Proxies can modify outgoing requests by adding or removing headers. (can also group multiple requests as one) This allows an organization to customize how traffic looks to the outside world or strip away sensitive information for better privacy.

Access Control: They enforce strict rules on who can access the internet. By requiring authentication and authorization, the proxy ensures that only permitted users or devices can reach external resources.

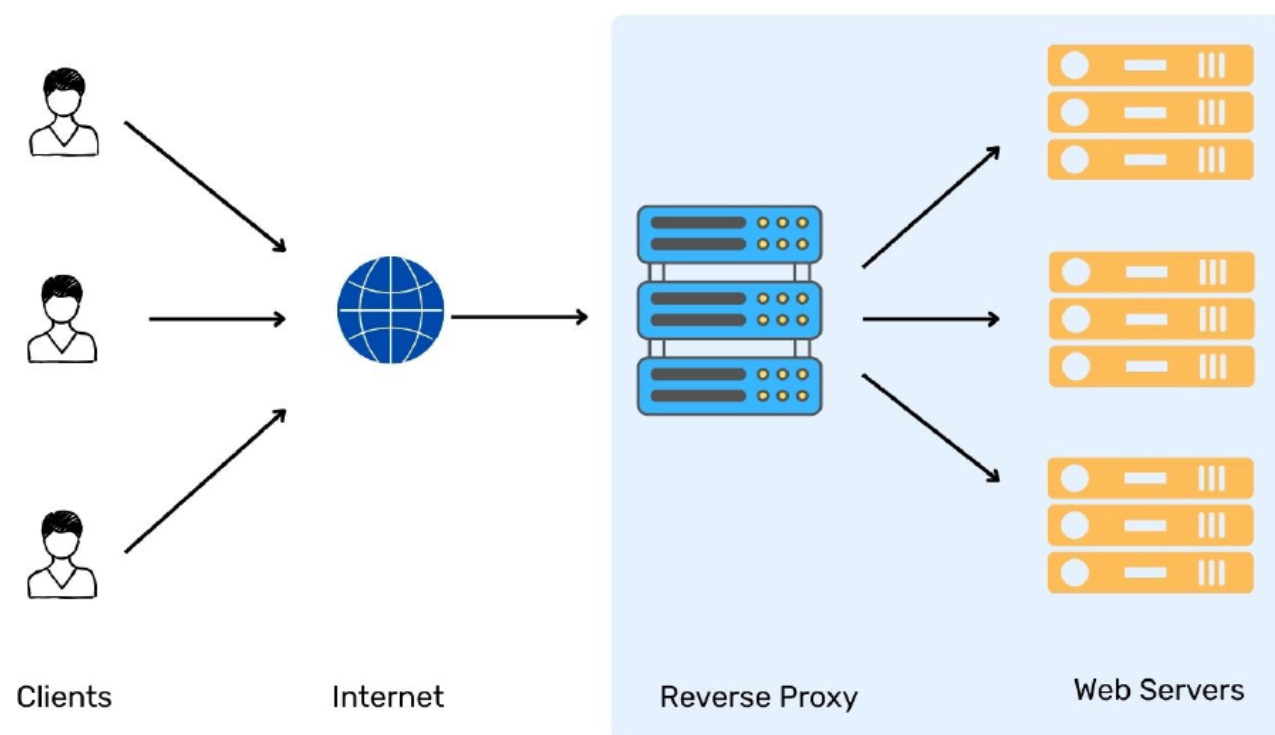
Real world example : I've seen this in my universities and corporate office. All internet traffic is routed through a proxy, so every request I make has to pass through that single checkpoint. If I try to access something that's blocked, say a restricted website, the proxy immediately detects it and cuts the connection based on predefined rules. This is how organizations, control what content is allowed within their networks.

Reverse Proxy (Server-Side Proxy)

A Reverse Proxy (Server-Side Proxy / internal-facing proxy) sits between the internet and the web servers (the destination). When a user makes a request to a website, the request hits the Reverse Proxy first. The proxy then decides which backend server should handle the request. Once the server finishes the job, it sends the data back to the proxy, which then passes it back to the user.

While a forward proxy acts as a middleman for the client, a reverse proxy acts as a middleman for the server. It hides the servers' real identity and protects them from the internet. It abstracts away the complexity of the downstream systems.

Reverse Proxy



Advantages of Reverse Proxy

Load Balancing: Reverse proxies act as traffic controllers. They distribute incoming requests across multiple servers so that no single server gets overwhelmed.

Server Protection & Security: The proxy acts as a shield. It hides the real IP addresses of the backend servers, making it impossible for hackers to target them directly. It can also block DDoS attacks or suspicious traffic before it even reaches the actual data.

SSL Termination: Handling the HTTPS lock icon (encryption) is heavy work for a server. The reverse proxy handles all the encryption and decryption, freeing up the backend servers to focus strictly on its tasks.

Caching: Reverse proxies can cache frequently requested resources, reducing the need for repeated requests to the web servers.

Response Modification & Compression: The proxy can shrink files (compression) to make them load faster on

your phone. It can also add or remove information from the server's response (headers) to make it more secure before it reaches the user.

Example :

An API Gateway is a specialized reverse proxy built for APIs

A load balancer (as it splits traffic across servers, it abstracts out the number of servers are there what they do) is a specialized reverse proxy for traffic distribution.

Db Proxies are specialized form of reverse proxy (sits in front of databases. It abstracts the database layer, handles connection pooling, and caches common query results.)

CDN (Content Delivery Network): A specialized, globally distributed reverse proxy. While a normal proxy sits in one data center, a CDN places edge proxies in hundreds of cities worldwide. It caches static content (like images and videos) as close to the user as possible to reduce latency.

Reverse Proxy can be all of these, but all of these can't be a reverse proxy.

Industry-Standard Reverse Proxies: Nginx, HAProxy, Kong Gateway, ProxySQL,

Thanks for reading Evolving Engineer! Subscribe for free to receive new posts and support my work.

Some must know differences when it comes to this topic would be:

Proxy vs VPN

VPN : A VPN creates a secure, encrypted tunnel for all the device's traffic (browsers, apps, system updates). It is secure because it encrypts your data, unlike proxies where it sends in plain text (unencrypted). (hides IP system-wide)

Proxy : It only works at the application level (e.g., just your web browser). It doesn't usually encrypt your data; it just masks your IP address for that specific app. It allows only for one app not system wide. (hides IP app-level)

Proxy vs Firewall

Firewall : firewall sits at the edge of the network and scans incoming and outgoing data packets (it only looks at the header of the data (IP and Port). It does not open the data. It can block/allow connections instantly based on rules, no need to terminate the connection), If the data is coming from a suspicious source, the firewall stops it at the door before it can get into your computer. So it just filters and blocks, it does not have proxy characteristics like caching.

Proxy : It is a middleman that terminates the client connection and starts a new connection to the server. Because it acts as the destination, it can "read" the data to perform caching or content filtering.

Hope you liked this article. If you did, please give it a like. If you have any questions or thoughts, drop a comment, this could spark a great discussion.

The next two articles are going to be about MVCC and my favorite Bloom Filters so stay tuned. And if you enjoy this kind of content, do follow me on LinkedIn and X for more.

And if you'd like to support my work, you can also buy me a coffee by clicking the link below, it really helps and means a lot.

buy me a coffee ?

Thanks for reading Evolving Engineer! Subscribe for free to receive new posts and support my work.

The Power of Bloom Filters in System Design

2026-01-31 · In this post, we'll break down Bloom filters from first principles and see how they're used in real-world systems.

Source: <https://pradyumnachippigiri.substack.com/p/the-power-of-bloom-filters-in-system>

Appreciate you reading this newsletter, I am writing as I learn and explore these topics in depth. I hope the content and resources are helping you level up as an engineer. If you want me to write about something specific, just message me or comment on the post.

Note: This article on was originally published on my personal website - <https://pradyumnachippigiri.dev/blogs/bloom-filters>

Here's what I learned this week

Reranking in RAG

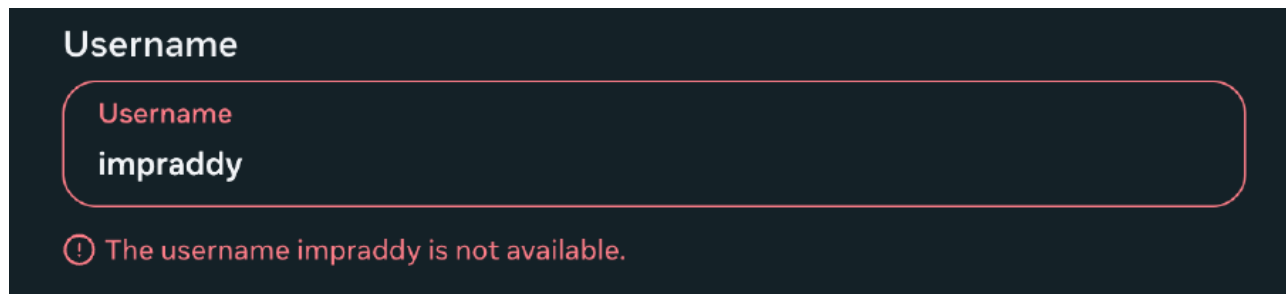
DORA metrics

Best LLM and Transformers Visualizations

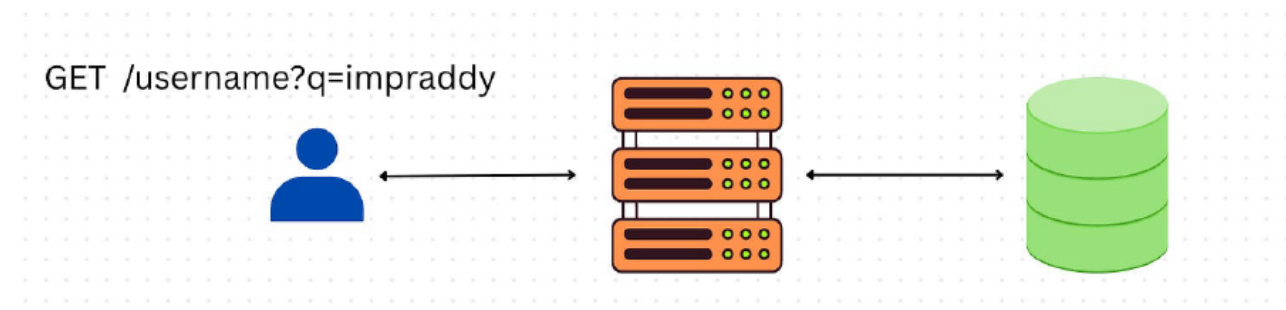
Let's start from Ground Up..

Say we're building an app, and as a part of the user registration process we ask users to choose a username. If the username is already taken, they need to try another one. (Just like your email, instagram usernames etc..)

So as engineers, how should we approach building a feature like this ?



The most straightforward approach that we can think of is that :



We will make the UI call an endpoint like this GET /username?q=impraddy

Our backend should check the cache/DB (SELECT * FROM users WHERE username = 'impraddy')

The endpoint should return True/False based on the availability.

then make the UI show the feedback instantly.

This works perfectly... at small scale.

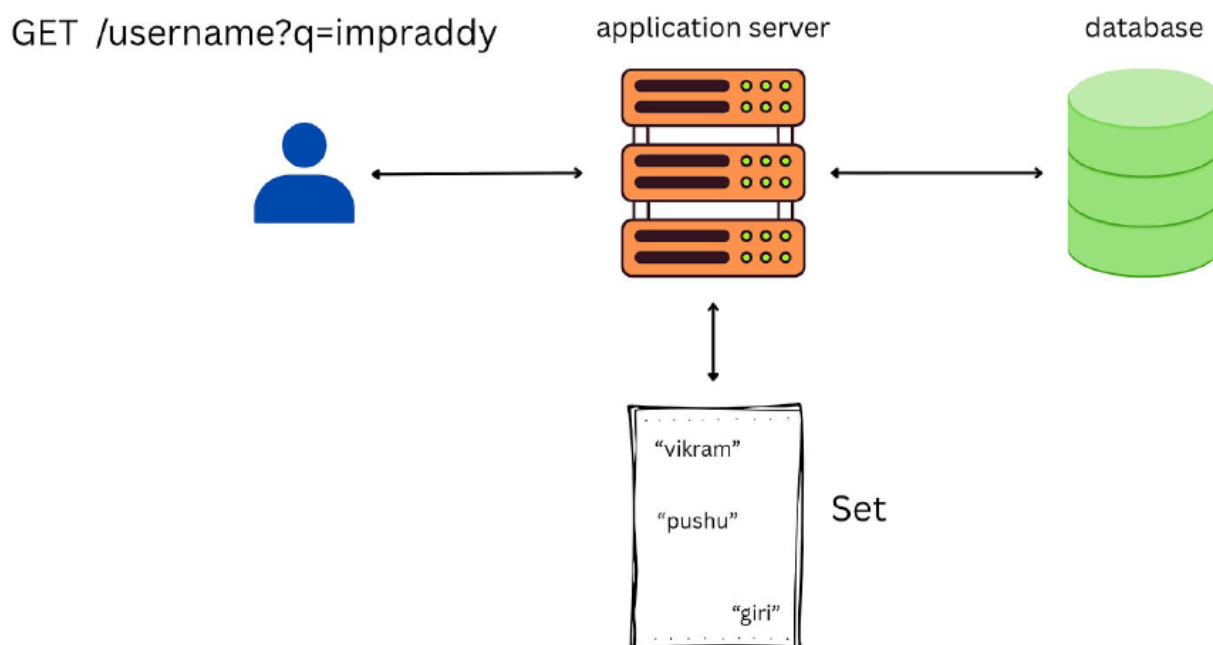
Now as the app grows, suddenly this endpoint becomes one of our hottest paths because this traditional approach does a full db scan which takes $O(n)$ which is good if the number of rows in our table is limited to some extent, but at millions and billions of records, the same query will take forever to return the result. (not only because we have multiple rows to check, but also that during registration people will retry multiple usernames, this in-turn increases the QPS, which can still hammer our DB, and increase the response latency.)

And introducing a cache doesn't make sense here as we are trying to see if a username exists or not. (cache stores the frequently accessed data, which is irrelevant to us).

So at this scale now we have to protect the db from being overwhelmed,

So our next approach would be to store the existing usernames in a set, and each time someone requests for a username check, instead of hitting the db, we will first check if the username is present in the set or no.

Thanks for reading Evolving Engineer! Subscribe for free to receive new posts and support my work.



A Set lookup is super fast as it gives a $O(1)$ look up time, but when our app grows to Instagram scale, storing billions of strings in a traditional in-memory Set will destroy our RAM, and also is super expensive. (1 billion usernames, each with average of 10 characters, roughly above 10GB + just for names ?)

Here is where Bloom Filters come in to the rescue, and solves the above problem optimally.

What is Bloom Filter ?

A Bloom Filter is a probabilistic data structure that answers membership queries like it checks whether :

the element is possibly present in the set, or

the element is definitely not present (100% certainty).

So this helps in solving questions like this efficiently : "Does the element exist in the set (data structure) or not?" For example see the real-world usecases below.

Bloom Filters Use cases

Instagram can use this to check if a reel was viewed by a user or no, so that they don't recommend the same reel to the user again

Medium can use this to check if the article was read by a user, and only recommend if the article was not read. (so that we don't recommend the same article twice.)

Web crawlers use this as well to check if a url was crawled before or no.

Not showing same movies as recommendations(for netflix) or avoid showing same ads to the user on any ads platform are also great examples where bloom filters fit perfectly.

It has numerous use-cases... Here's another discussion from LinkedIn from an Amazon Employee as to how they used Bloom filters...



Karthik Subramanian ✓ • 1st
Engineering Leader & Bar Raiser | Cloud...
4m • 🌐



Bloom Filters are gorgeous!!

I recently saw a post from [Carlos Arguelles](#) about Bloom filters, which took me back to the times I have used it in Amazon.

In one instance, while working on Alexa availability, we were focussed on cutting down on redundant DDB and service calls. Every "Play music" request required multiple calls to a downstream to check for credentials (Spotify, Radio, library etc.), even though most users didn't have those accounts linked. [Pavan Kumar Reddy N](#) and I designed an in-memory Bloom filter to answer the question in-memory when not to call for credentials. It was so elegantly simple and it allowed us to bypass the dependency entirely most (99%) of the times. The real challenge was making the Bloom filter bitmap updates happen in the background that the customer never felt a delay when they actually linked an account.



Carlos Arguelles ✓ • 1st
Senior Principal Engineer, Engineering Productivity at Amazon (ex-Google...
2d • Edited • 🌐



As a comp sci nerd, I'm always excited when somebody brings up a **bloom filter** as a solution to a real-life problem at work 🙌 -- we did yesterday and it's actually a really elegant solution to an annoying problem we were facing.

A Bloom Filter is a space-efficient, probabilistic data structure used to test whether an element is a member of a set.

Think of it as a "fast-pass" check. It can tell you with 100% certainty if something is not in a set, but it can only tell you if something is likely in a set.



Like



Comment



Repost



Send

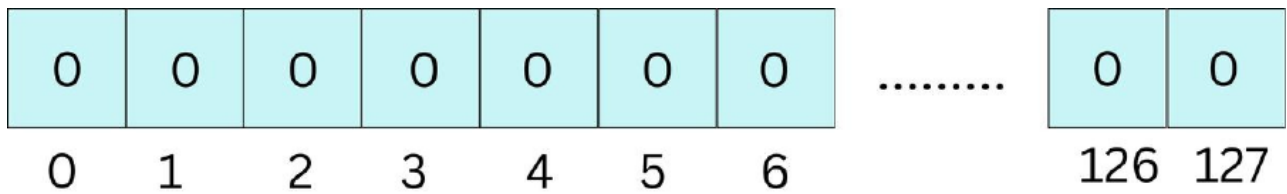
Now let's understand how bloom filters work..

Thanks for reading Evolving Engineer! Subscribe for free to receive new posts and support my work.

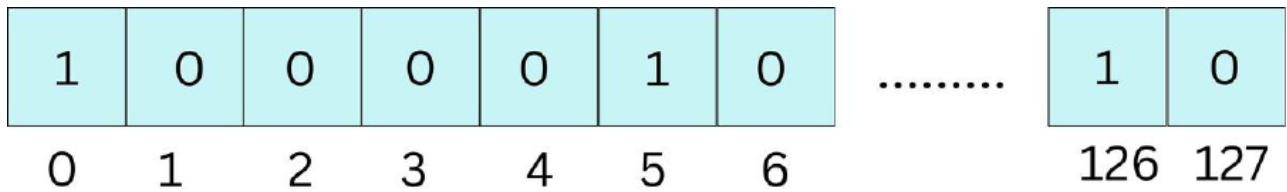
How does Bloom filter Work?

Let's understand by taking an example :

Imagine we have an array of 128 buckets where each bucket consists of a single bit that is set to zero. Since it's a bit, it can only have two values: either 0 or 1. Initially, all 128 bits are set to 0.

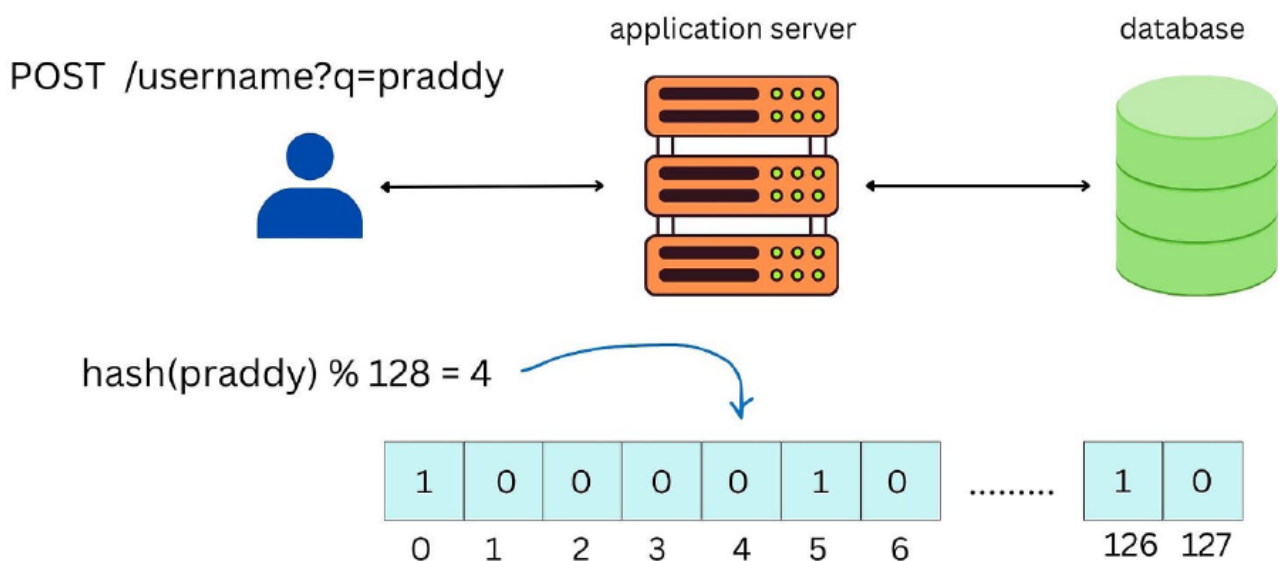


Let's consider the previous example itself, the example of finding a username.. (for better understanding let's say we already have 3 users "pushu", "giri" and "vikram", and we have marked their hash values to 1.)



Insert Operation

When the user is typing a username, we might do a GET availability check in the bloom filter (refer to the query operation section below).



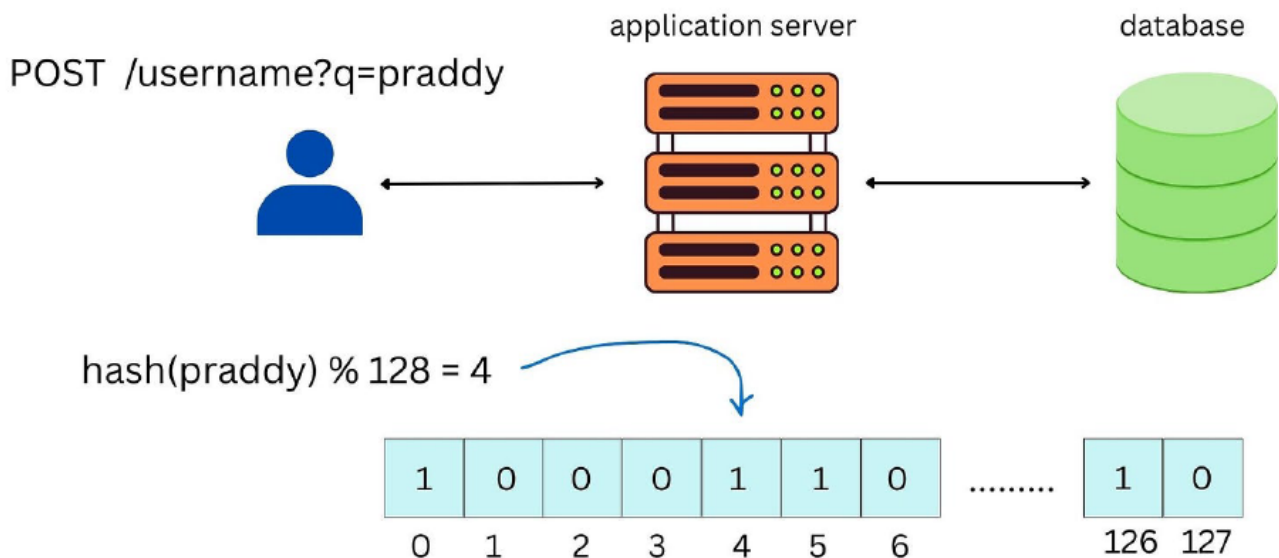
But the insert happens when the bloom filter query operation (GET) returns a 0 (100% certain that the user is not present) and only when the user finally clicks Sign Up and we create the account via a POST request. Only after the DB insert succeeds, we update the Bloom filter like below.

For insert operation we need to perform the following operations :

Take the hash value of the username

then take the hash and mod it with the number of bits considered in the bit array. so in our case it becomes $\text{hash}(\text{username}) \% 128$

Set the corresponding bit ($\text{hash}(\text{username}) \% 128$) to 1 in the array. If a bit is already 1, we just leave it as 1.



Query/Read Operation

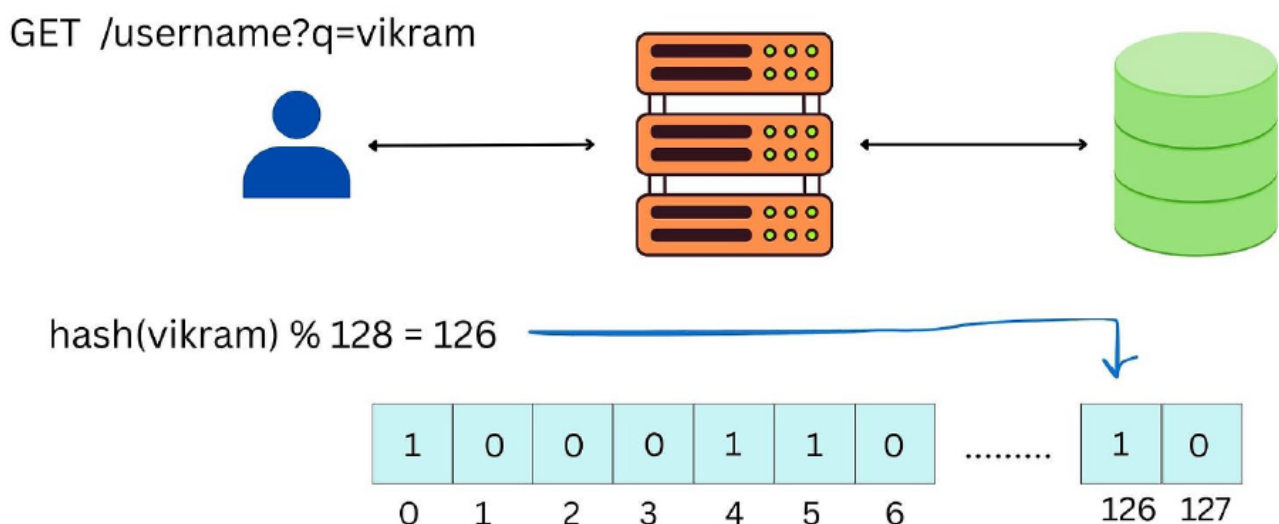
For query operation (meaning to see if a username exists or not)we perform the following steps :

Take the hash value of the username we are checking

mod it with the total number of bits in the array : $\text{hash}(\text{username}) \% 128$ in our case

then we check if the bit at that index is set or not.

If the index is not set (meaning it is 0), then it is 100% certain that the username is not registered. But if the index is set to 1, then it means that the username may or may not be registered, so we have to make that db call to know if it really exists or no.



Some Important Points to Note

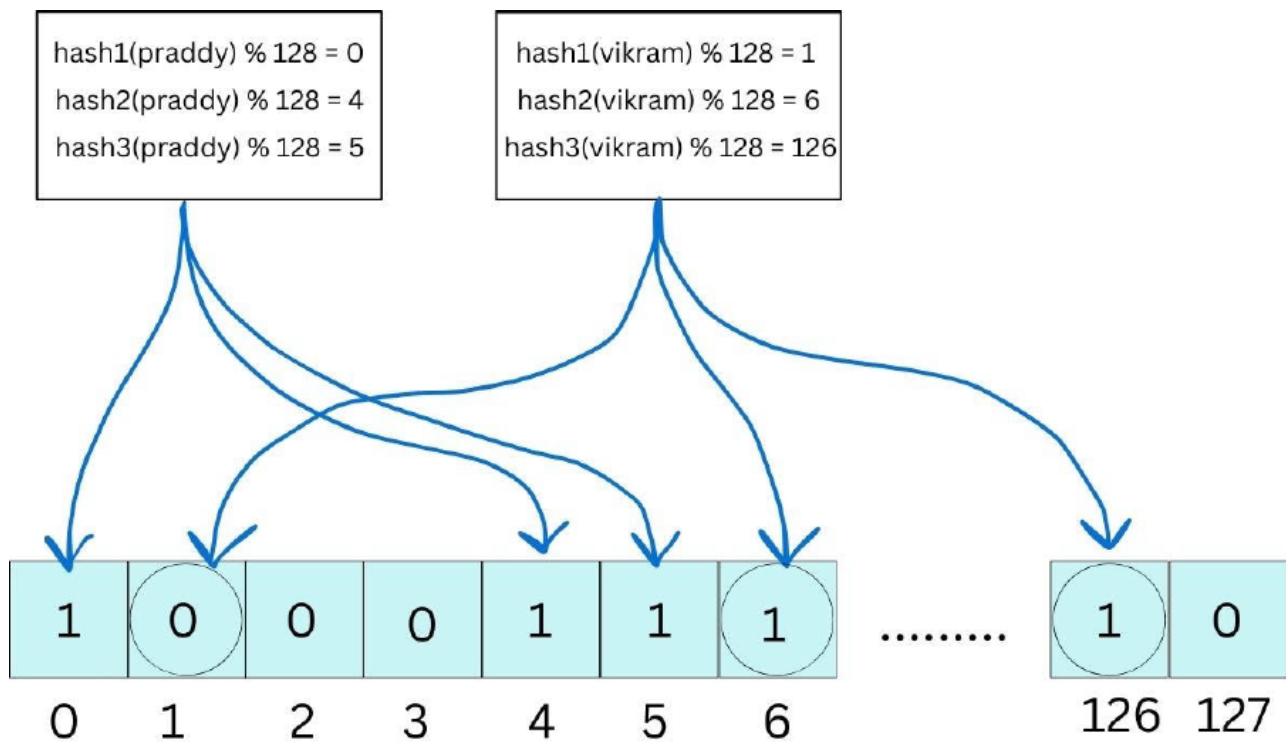
In the query operation, the “may or may not be present” case introduces false positivity, meaning bloom filters can say taken (the set bit ‘1’) even when it’s actually not taken (so we are hitting the db for such cases). This is called the false positivity rate, p .

Number of keys we put in bloom filters increase, the false positivity rate increases

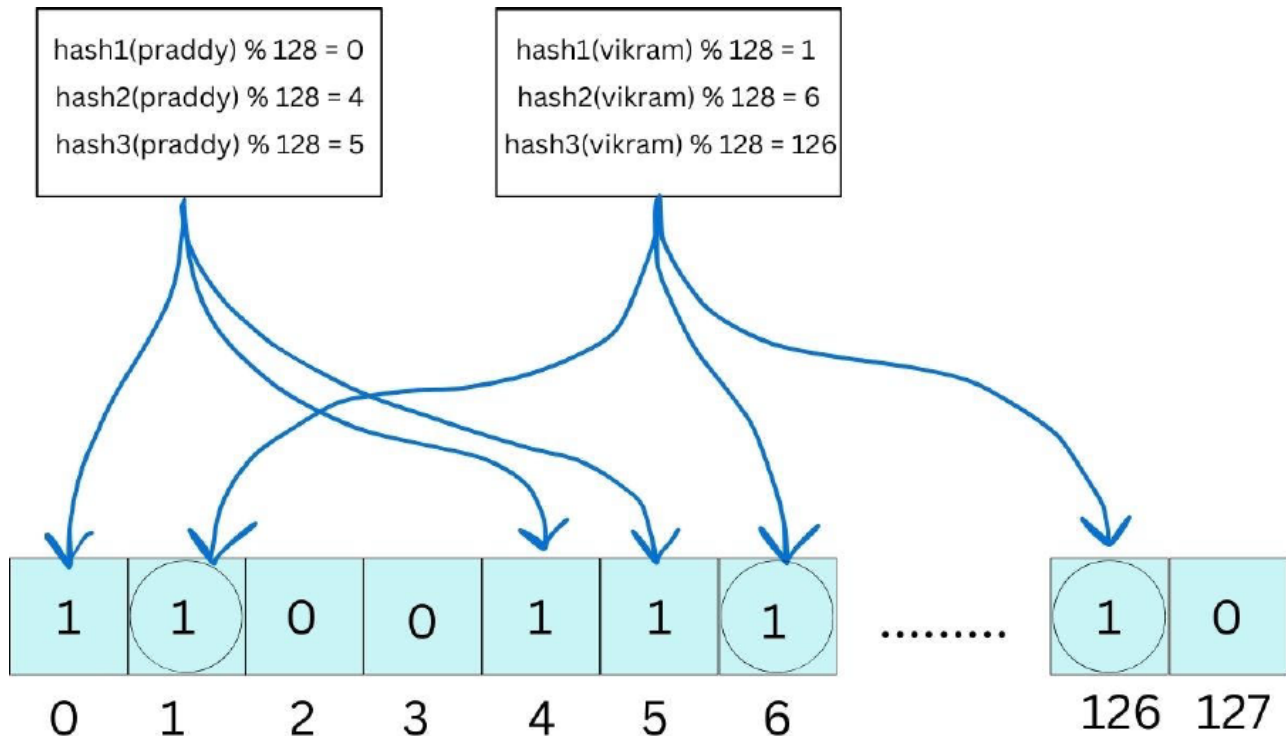
In the above example, we considered only 1 hash function. But in real systems, we use k hash functions,

meaning we set/check k bits per input (username in our case):

If either of the k bits is 0 -> definitely not present (in the diagram below, "vikram" is definitely not present)



If all k bits are 1 -> maybe present (false positives possible, query the db)



Thanks for reading Evolving Engineer! Subscribe for free to receive new posts and support my work.

Obvious Questions that Pop-up

What happens if all the boxes end up with 1s ?

Well, in that case the bloom filter becomes useless, because for every query it'll say "maybe present", and we'll end up hitting the DB every time, this in-turn increases the false positivity rate (which is not what we want) so we lose the whole benefit of using it.

That's why sizing matters: we need enough bits (and the right number of hash functions) so the filter doesn't get too full, and the false positive rate stays low.

So what do we do when the filter starts getting full? Can we scale it?

The common approach is to layer Bloom filters (the idea behind scaling bloom filters).

When the current Bloom filter starts getting too full (too many 1s -> false positives go up), we don't try to expand the same bit array (because we would need to rehash and reorder , not optimal). Instead:

We freeze the current Bloom filter (keep it as-is)

We create a new Bloom filter (usually bigger / tuned better)

We insert all new elements into the new one

For query(element) operation, you might have to check in all instances of Bloom Filters and then take a decision accordingly.

So how do we decide the number of bits to choose, and the number of hash functions to choose ?

There's a lot of math involved in this one, and if you are curious to learn on how to choose the optimal k and m, you can check these 2 wonderful articles out (usually u just use those formulaes directly, no need to remember them):

<https://arpitbhayani.me/blogs/bloom-filters/>

https://en.wikipedia.org/wiki/Bloom_filter#Probability_of_false_positives

<https://nyadgar.com/posts/the-beautiful-math-of-bloom-filters/>

What if a username/user (the element) is deleted?

Classic Bloom filters don't support deletes. A variant of bloom filters called the Counting bloom filters supports deletion. There are multiple variants of bloom filters, each of them has its own use case, you can always explore them..

When should i use a bloom filter ? What patterns to identify ?

We keep doing existence checks: "does X exist?" / "have we seen X before?"

A DB/Redis/disk lookup for "NO" is expensive or happens at high QPS

Use Bloom filters when we only insert data and never remove it.

Use when we specifically need to answer "NO" with 100% certainty.

We have finally come to an end of this long deep dive. Also, I have implemented Bloom Filters in Python from First Principles that is starting from brute force to production grade optimal approach.

I will be talking more about the code step by step in my coming articles..

If you liked this article, feel free to like, subscribe, and drop a comment to spark a discussion, so we can all learn from each other. And if you think it'll help someone, share it with your friends or on social media.

If you'd like 1:1 help (system design, backend, DSA, resume review etc.), you can book a session with me on Topmate

If you found this useful and want to support my work, you can also buy me a coffee ?
buymeacoffee.com/cpradyumnao

Thanks for reading Evolving Engineer! Subscribe for free to receive new posts and support my work.

Share

Apache Kafka Basics

2026-02-09 · Understanding the core components of Kafka for System design interviews

Source: <https://pradyumnachippigiri.substack.com/p/apache-kafka-basics>

Appreciate you reading this newsletter, I am writing as I learn and explore these topics in depth. I hope the content and resources are helping you level up as an engineer. If you want me to write about something specific, just message me or comment on the post.

Note: This article on was originally published on my personal website - <https://pradyumnachippigiri.dev/blogs/kafka-fundamentals>

Here's what I learned this week

uv vs pip

p50, p95, p99-latencies

explain vs explain analyze

This article is my attempt to make Kafka feel intuitive and easy to understand, we'll walk through the core building blocks of Kafka with solid reasoning behind the design decision.

Let's dive in..

What is Kafka ?

Kafka documentation states that "Apache Kafka is an open-source, highly scalable, durable, fault-tolerant distributed event streaming platform". Variety of companies use Kafka in production systems, you can access the list of companies that use Kafka [here](#).

Stream is a continuous flow of data (that we call records/messages/events). These messages are published by the producers and the consumers read and process the messages. Kafka/any streaming platform handles this workflow efficiently by possessing the following capabilities :

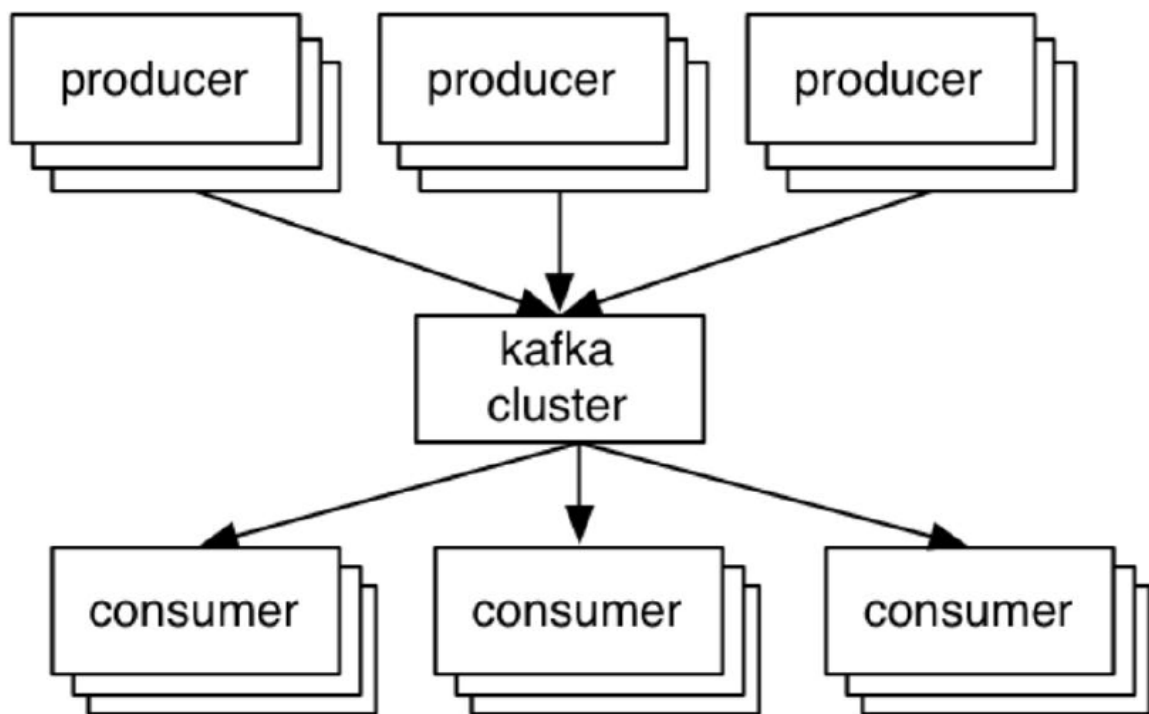
Collect the data sent from multiple producers, and store them.

Make sure the stored messages are not lost even during outages/failures.

Be scalable incase of heavy load.

Exhibit Fault tolerant behaviour

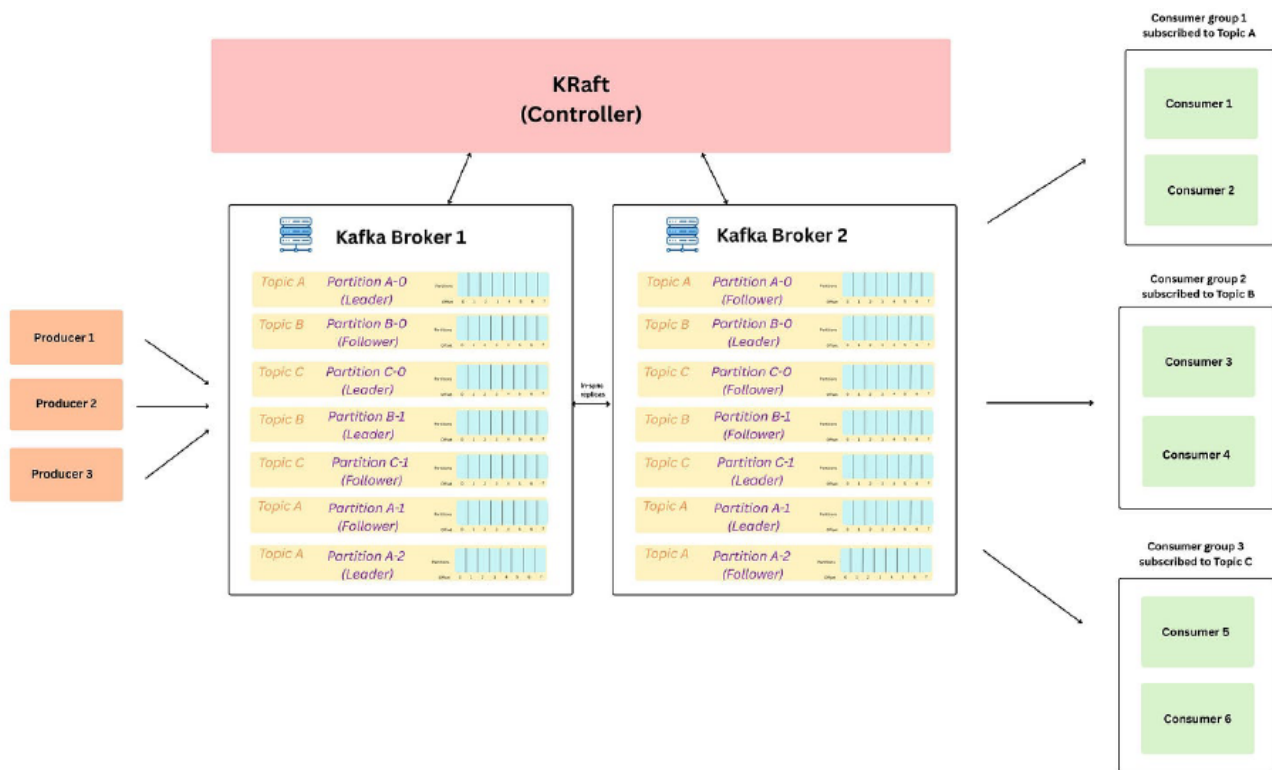
Enable access to multiple consumers



In a nutshell, Kafka works on the traditional client-server architecture. The server is Kafka, the producer and the consumer are the clients. The communication between client and server happens via customized high-performance TCP protocol implemented by the Kafka team. There are client libraries available for all most all programming languages.

Kafka Architecture

On a higher level, this is how the kafka architecture looks like, in the sections below we will go over all the key components.. (Keep referring back to this image, incase you want a visual understanding of any component, as to where a block sits.)



Message Structure

A message (also called data / record / event) is the basic unit of data stored inside the Kafka log. (which we will discuss in sometime..) We can think of a message like a row in a database.

This message/event represents something that happened. (like a trigger for async processing)

Metadata(unique id)

Headers

Timestamp

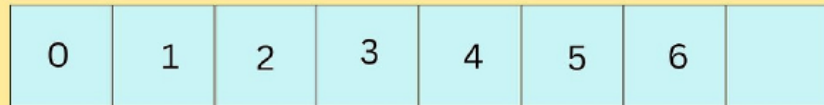
Key → value

An event describes something that happened. For example, in an e-commerce system, examples of events can be “an order is created”, “an item is added to cart”, “an order is delivered”. In Kafka, we read or write data in form of events. We can send this data in XML, JSON or Avro

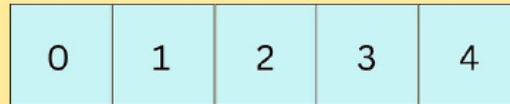
Each message is nothing but a key-value pair, which looks like this.

Topics

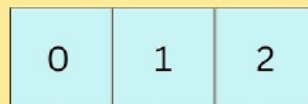
Partition 0



Partition 1



Partition 2



•
•
Topic
"abc"

In a real system, there are multiple kinds of events flowing continuously like orders, payments, application logs, etc. It makes sense to group these related events together, and in Kafka this grouping is called a topic.

So Topic is basically just a logical abstraction: it is a named stream that groups related records together.

We can think of kafka topics like a table in a database (collection of related records), or a folder in a file system (collection of related files)

Every Kafka message is published to exactly one topic. We can have multiple topics in our Kafka cluster each storing a specific category of events. Topics are the way you publish and subscribe to data in Kafka. When you publish a message, you publish it to a topic, and when you consume a message, you consume it from a topic. Topics are always multi-producer; that is, a topic can have zero, one, or many producers that write data to it.

Internally, each topic is a combination of multiple partitions.

Thanks for reading Evolving Engineer! Subscribe for free to receive new posts and support my work.

Partitions

Hussein Nasser always quotes that "the best way to work with billions of rows in a database is to avoid working with billions of row." That's exactly how even Kafka works.

Just like how a database shards data when a single node becomes a write bottleneck, Exactly how even kafka scales write throughput by increasing the number of partition's in a topic, this allows the writes to happen in parallel.

Partition is nothing but an ordered, immutable, append-only log of messages (Just like write ahead logs in db) Since the messages are always appended, they can be read in the same order from beginning to end.

Partitioning gives kafka, the ability to scale out (meaning, partitions can spread across cluster of machines) and hence now it can hold more data than how much a single machine could have handled. Since, its partitioned,

everything like storage, reads, and writes gets distributed across the cluster: producers can write to multiple partitions in parallel, and consumers can read from multiple partitions in parallel.

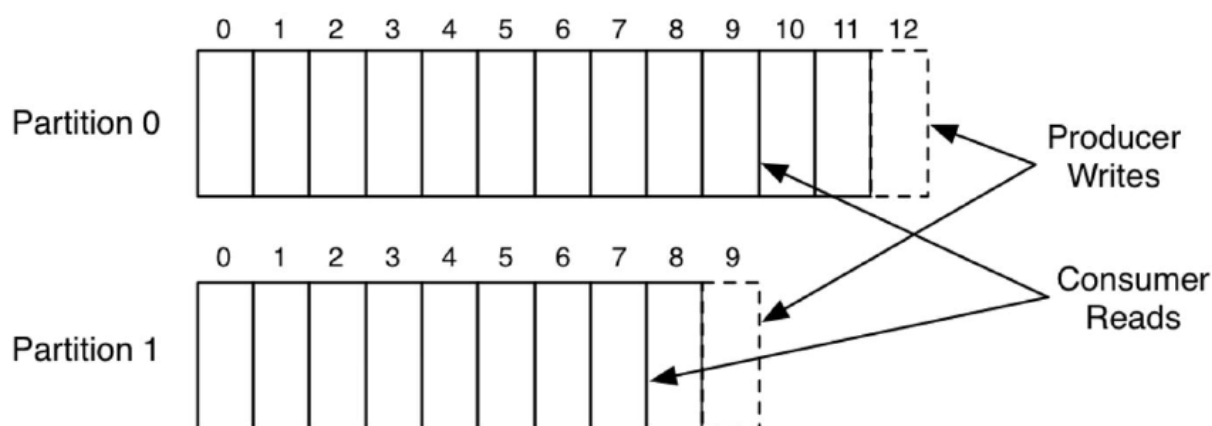
Each partition contains a different subset of the topic's data meaning if we take the union of records across all partitions, we get all the records that belong to the topic.

Offset : Offset is a unique identifier assigned to each message within a partition. It like the position number of that message in the log (0, 1, 2, ...).

We will talk about why offsets are useful in the COnsumers section below.

Important note is that these logs are persistent meaning, every message received is that immediately written to the disk.

Below is a simple example of a single producer, and a single consumer writing and reading from a 2-partition topic, borrowed the image from LinkedIn's Engineering Blog



Kafka Broker

Kafka brokers are basically the servers that run Kafka. We talked about how producers publish data to a topic, and consumers consume from the topic. We also talked about how the data is appended into an append-only log inside partitions. But where does all of this actually live and run?

That's where brokers come in.

When a producer publishes an event to a topic, Kafka automatically routes that event to the correct partition of that topic. That partition is stored on some broker, and the broker is responsible for receiving the write request and persisting the event to disk. Similarly, when a consumer subscribes to a topic, it reads the data by fetching messages from the brokers that host the partitions for that topic.

A Kafka cluster is simply a group of multiple brokers working together. Having multiple brokers allows Kafka to distribute partitions across machines, handle high traffic, and stay available, even if one broker goes down. So by adding more brokers, Kafka can scale horizontally without changing producer or consumer code.

Thanks for reading Evolving Engineer! Subscribe for free to receive new posts and support my work.

Producers

A producer is a client application that publishes events to Kafka topics. Producers send records to a topic, and Kafka routes those records to the appropriate partitions, which are stored on brokers.

Question : We said “producers send data to a topic” and “data ends up in a partition on a broker” but how does Kafka decide which partition a record should go to? And can producers control that?

Answer:

Kafka follows a few simple rules:

If the producer does not provide a key, Kafka distributes records across partitions to balance load (commonly round-robin or sticky partitioning depending on the client).

If the producer does provide a key, Kafka uses that key to consistently choose a partition.

Under the hood, this is usually:

Partition no = $\text{hash}(\text{key}) \% p$, where p = number of partitions

Messages with the same key always go to the same partition, which means they are stored in order within that partition

If we decide to change the number of partitions p later (increase/decrease), the same key may start routing to a different partition, so there might not be ordering guarantee. (so its always adviced to choose the partition number upfront based on the parallelism and throughput needs.)

If needed, producers can also:

Directly specify the partition number, or

Implement a custom partitioning logic (useful, but should be done carefully to avoid hot partitions and uneven load)

I'll write a separate article in the coming weeks on the internals of the Kafka Producer. There's a lot happening behind the scenes when you call the Producer API, and understanding it makes it much easier to reason about scaling, idempotency, retries, and delivery guarantees.

Consumers

A consumer is a client application that subscribes to one or more Kafka topics and reads records from them. Internally, consumers always read from partitions, not from a topic as a whole. They work on a pull based model.

here's a wonderful write up from this page of Kafka documentation, as to why they chose pull over push for consumers.

Push vs. pull

An initial question we considered is whether consumers should pull data from brokers or brokers should push data to the consumer. In this respect Kafka follows a more traditional design, shared by most messaging systems, where data is pushed to the broker from the producer and pulled from the broker by the consumer. Some logging-centric systems, such as [Scribe](#) and [Apache Flume](#), follow a very different push-based path where data is pushed downstream. There are pros and cons to both approaches. However, a push-based system has difficulty dealing with diverse consumers as the broker controls the rate at which data is transferred. The goal is generally for the consumer to be able to consume at the maximum possible rate; unfortunately, in a push system this means the consumer tends to be overwhelmed when its rate of consumption falls below the rate of production (a denial of service attack, in essence). A pull-based system has the nicer property that the consumer simply falls behind and catches up when it can. This can be mitigated with some kind of backoff protocol by which the consumer can indicate it is overwhelmed, but getting the rate of transfer to fully utilize (but never over-utilize) the consumer is trickier than it seems. Previous attempts at building systems in this fashion led us to go with a more traditional pull model.

Another advantage of a pull-based system is that it lends itself to aggressive batching of data sent to the consumer. A push-based system must choose to either send a request immediately or accumulate more data and then send it later without knowledge of whether the downstream consumer will be able to immediately process it. If tuned for low latency, this will result in sending a single message at a time only for the transfer to end up being buffered anyway, which is wasteful. A pull-based design fixes this as the consumer always pulls all available messages after its current position in the log (or up to some configurable max size). So one gets optimal batching without introducing unnecessary latency.

The deficiency of a naive pull-based system is that if the broker has no data the consumer may end up polling in a tight loop, effectively busy-waiting for data to arrive. To avoid this we have parameters in our pull request that allow the consumer request to block in a "long poll" waiting until data arrives (and optionally waiting until a given number of bytes is available to ensure large transfer sizes).

You could imagine other possible designs which would be only pull, end-to-end. The producer would locally write to a local log, and brokers would pull from that with consumers pulling from them. A similar type of "store-and-forward" producer is often proposed. This is intriguing but we felt not very suitable for our target use cases which have thousands of producers. Our experience running persistent data systems at scale led us to feel that involving thousands of disks in the system across many applications would not actually make things more reliable and would be a nightmare to operate. And in practice we have found that we can run a pipeline with strong SLAs at large scale without a need for producer persistence.

Question : if a topic has multiple partitions and you spin up multiple consumers, how does Kafka decide who reads what, and how does it avoid two consumers accidentally processing the same message (and avoid duplicate processing while preserving ordering)?

Answer:

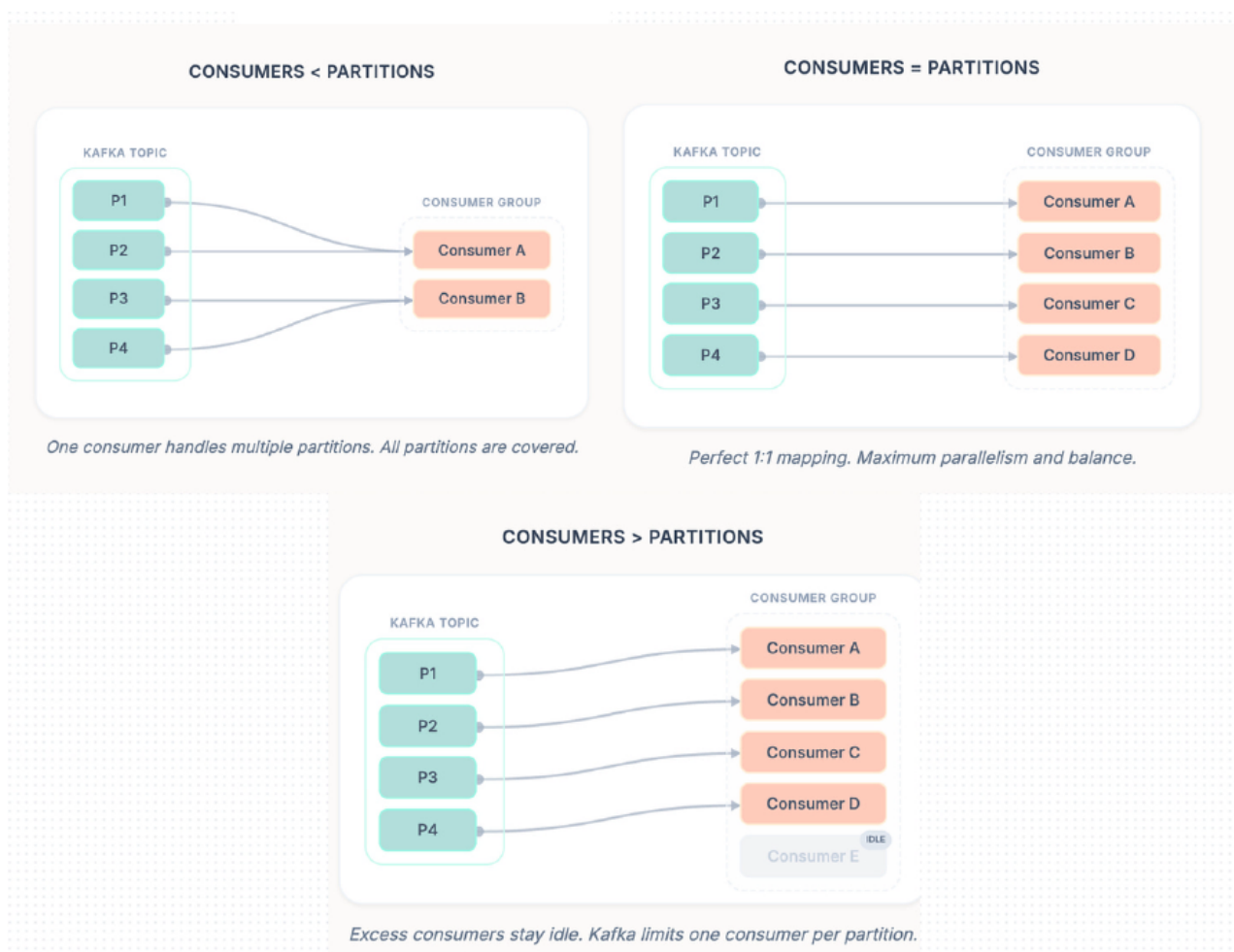
If consumers were allowed to read partitions independently without any coordination, then every consumer could end up reading the same partitions and hence the same messages, leading to duplicate processing.

Kafka solves this by introducing consumer groups. A consumer group is just a logical grouping of consumers identified by the same group.id. When multiple consumers join the same group, Kafka treats them like a team that wants to share the work, and it enforces a very important rule:

each partition is assigned to at most one consumer within the group

A consumer can read from one or more partitions, but no two consumers in the same group will read from the same partition. This keeps ordering intact (since each partition is an ordered log) and ensures that each message is processed only once per consumer group. (but different consumers in different consumer groups can consume from those partitions..).

So since consumers in a consumer group are consuming from partitions, here's how they are related.:



If number of consumers in a consumer group < number of partitions of the topic it is consuming from -> each consumer is assigned multiple partitions

If number of consumers in a consumer group == number of partitions of the topic it is consuming from -> each consumer is assigned exactly one partition

If number of consumers in a consumer group > number of partitions of the topic it is consuming from -> extra consumers remain idle (for that topic)

If a consumer group subscribes to multiple topics -> this assignment logic is applied independently per topic

Now remember the offset we briefly introduced earlier as the position number (0, 1, 2...) of a message inside a partition, this is exactly why offsets are useful. As consumers read from a partition, Kafka tracks how far they have read using offsets, and these offsets are tracked per consumer group, per partition. Kafka stores these committed offsets inside an internal Kafka topic called `__consumer_offsets`, which is how Kafka can remember progress even if a consumer crashes or restarts. Because of this, consumers can resume from where they left off after failures, and different consumer groups can consume the same topic independently with their own progress

I will dedicate a separate article to discuss on the internals of kafka Consumers

Replication

Till now, we talked about how partitions store data on brokers. But if a partition existed on only one broker, Kafka would not be fault tolerant, if that broker went down, data would be unavailable or even lost right. Kafka solves this using replication.

Leader Replica: Each partition has one replica elected as the leader, and that leader lives on a broker. Producers and consumers always talk to the leader for that partition (writes and reads happen via the leader).

Follower Replicas: The same partition is replicated onto other brokers as followers. Followers don't serve client traffic, they passively replicate data from the leader.

Distributed Leadership: In the architecture diagram above, notice how leaders are spread across brokers (ex: Partition A-0 leader on Broker 1, Partition A-1 leader on Broker 2). This is intentional so read/write load is distributed and no single broker becomes a hotspot.

ISR (In-Sync Replicas): Followers that are fully caught up with the leader are part of the ISR. ISR is critical because only an in-sync follower can safely be promoted as the next leader without losing acknowledged data.

Failover: If the broker hosting a partition leader goes down, Kafka promotes one of the ISR followers as the new leader, so producers and consumers can continue operating with minimal downtime. When the failed broker returns, it usually rejoins as a follower and starts replicating again.

At this point, we understand what replication is and why Kafka uses leaders, followers, and ISR. But this naturally raises the next question: who tracks broker liveness, who decides leaders, and who coordinates failover and metadata updates? That responsibility belongs to Kafka's control plane the KRaft Controller.

KRaft Controller

Kafka replication and failover only work if the cluster has a single, consistent authority that coordinates metadata changes. That role is handled by the Kafka Controller (Kafka's control plane).

At a high level, the controller's responsibilities include:

Leader assignment: deciding which replica is the leader for each partition

Managing replication state: tracking replicas and maintaining the ISR set

Broker health monitoring: detecting when brokers go down or come back up

Failover coordination: when a leader broker fails, promoting an in-sync follower as the new leader so the partition stays available

Earlier versions of Kafka used ZooKeeper for this coordination. Modern Kafka replaced ZooKeeper with KRaft (Kafka Raft), where Kafka manages this metadata internally (no external dependency).

One important note: the controller doesn't handle data traffic, producers and consumers don't talk to KRaft directly. KRaft just coordinates the cluster/brokers in the background so Kafka keeps working correctly even as brokers come and go.

Well, this article was covering the fundamentals of Kafka, In the coming months, we'll go deeper into the parts that matter when you actually run Kafka in production:

Kafka Producer Internals : Deep dive into the Producer API

Kafka Consumer Internals : Deep dive into how Consumer API works

End to End delivery semantics : At-least-once vs at-most-once vs exactly-once, how duplicates happen, how idempotency and transactions work, and what guarantees Kafka can (and cannot) provide.

Thanks for reading Evolving Engineer! Subscribe for free to receive new posts and support my work.

If you liked this article, feel free to like, subscribe, and drop a comment to spark a discussion, so we can all learn from each other. And if you think it'll help someone, share it with your friends or on social media.

If you'd like 1:1 help (system design, backend, DSA, resume review etc.), you can book a session with me on

Topmate

If you found this useful and want to support my work, you can also buy me a coffee ?
buymeacoffee.com/cpradyumnao
